Software:

- Software is a collection of specialized program which takes user input and generate desired output.
- A Software is a collection of computer programs that helps us to perform a task.
- It is a way to assess the quality of the software and to reduce the risk of Software failure in operation.

Types of Software:

1) System software

- Ex: Device drivers, Operating Systems, Servers, Utilities, etc.

2) Programming software

- Ex: compilers, debuggers, interpreters, etc.

3) Application software

- Ex: Web Applications, Mobile Apps, Desktop Applications etc.

Types of Applications:

1) Web Applications :

- Run in a browser (Chrome, Firefox, and Edge).
- Examples: Gmail, Facebook, Amazon etc.
- **Testing Focus:** Cross-browser, responsiveness, security, performance.

2) Mobile Applications :

- Installed and run on mobile devices.

a. **Native Apps:**

- **Development:** built specifically for a single platform (example iOS or android). Using specific programming language and tool (example Swift/Objective-c for iOS , kotlin/Java for android)
- **Platform Compatibility:** Run only on the platform it was built for.
- **Device Access :** Full, direct access to all device hardware and software features (example : camera , GPS , Push Notification)
- **Performance :** Generally offers the best performance and user experience due to optimization for the specific platform
- **Example:** The default camera app on the iPhone or an android device.

b. **Web Apps:**

- **Development:** built using web technologies like HTML, CSS and JavaScript. Accessed through a web browser
- **Platform Compatibility :** Run on any device with a compatible web browser , regardless of the operating system
- **Device Access:** Limited access to device features compared to native or hybrid app, primarily restricted by browser capabilities.

- **Performance :** Performance depends on the browser or internet connection: typically not as performant as native app
- **Example**: Google Docs or a banking website accessed through a mobile browser.

c. Hybrid Apps:

- **Development:** Combines elements of both native and web apps. The core logic is built using web technologies (HTML, CSS, and JavaScript) and then wrapped in a native container (like CoreDova or Capacitor) that allows it to be distributed through app stores and access some native device features.
- **Platform Compatibility:** Can run on multiple platforms (iOS and Android) from a single codebase.
- **Device Access:** Can access some native device features through plugins or bridges, but may not have the same level of direct access or performance as a truly native app.
- **Performance:** Generally performs better than a pure web app but may not match the seamless experience of a native app.
- **Example**: Many social media app or e-commerce app that use web views for certain content while leveraging native features like push notifications. Instagram, has used a hybrid approach for certain functionalities.

3) **Desktop Applications**
   - Installed directly on PC/Laptop.
   - Examples: MS Word, Photoshop, Notepad++.
   - **Testing Focus:** Installation, OS compatibility, resource usage.

4) **Client-Server Applications**
   - Have a **client app** (frontend) and a **server app** (backend).
   - Example: Banking apps, chat apps.
   - **Testing Focus:** Database, concurrency, network, integration.

5) **Enterprise Applications**
   - Large-scale apps used by organizations.
   - Examples: ERP, CRM (like Salesforce, SAP).
   - **Testing Focus:** Integration, workflows, data security, and role-based access.

6) **Cloud Applications**
   - Hosted on cloud, accessed via web.
   - Examples: Google Drive, Dropbox.
   - **Testing Focus:** Scalability, data security, disaster recovery.

What is software testing?

1. Software testing is a part of software development process.
2. Software testing is an activity to detect and identify the defects in the software.
3. The objective of testing is to release quality product to the client.
4. It is process of checking the correctness and completeness of the software with respect to the client expectation.

Software Quality Depends Upon:

1. The product should bug - free.
2. The product should delivered on time.
3. The product should be within budget.
4. The product should meets requirements and expectations.
5. The product should maintainable.

Objectives of Testing:

1. To prevent defects by evaluate work products such as requirements, user stories, design and code
2. To verify whether all specified requirements have been fulfilled.
3. To check whether the test object is complete and validate if it works as the users and other stakeholders expect.
4. To build confidence in the level of quality of the test object.
5. To find defects and failures thus reduce the level of risk of inadequate software quality.
6. To provide sufficient information to stakeholders to allow them to make informed decisions, especially regarding the level of quality of the test object.
7. To comply with contractual, legal, or regulatory requirements or standards.
8. The product should maintainable.

Why Software has the Bugs?

1. If there is a Miscommunication or no communication between development and testing team.
2. Software complexity
3. If no of programming errors that result software have more defects.
4. If the frequently changing requirements from client.
5. Lack of skilled testers.

What are the 3 pillars of any company?

a. P-People.
b. P- Process.
c. P- Product.

Difference between Product and Project:

a. If software application is developed for specific customer based on the requirement, then it is called **as project.**

b. If software application is developed for multiple customers based on market requirements, then it called **as product.**

What is prototype?

1. Prototype is nothing but the visualization of functionality before development.
2. When the requirement came from client, business analyst prepare a prototype model to show them(client).
3. After approval from client they will work on it as design, development, coding and testing.

4. Then build or application is deploy to the client side.
5. It is made with the help of Rally Tool.

Manual Testing:

1. **Defines:** Check/Validation developed application will work as per the requirements.
2. Check completeness and correctness of the functionality by manual.

Software Quality Assurance:

1. SQA process defines to measure and monitor the developed application.
2. SQA define software quality assurance.
3. It is communication between business analyst and client.

SQA process parameters:

a. Clients/ customer requirements full fill :

- What is the purpose of the project?
- Which type of application customer want? (Web Based+ Mobile Base+ Domain)
- Domains: Banking/Telecom/Insurance/Pharma/Ecom/Travel etc.

b. Clients/ customer expectation (security & performances):

- **Security/Privacy:** Let's consider an example for the banking domain, Software like banking gathers a lot of customer data which is very sensitive, so, in that case, clients want to security/privacy with respect to the all data.
- **Performance:** Software/Application should work properly under the extreme load, Load balancing working proper.
- **Maintenance:** Need to take care, If any problem occurs after delivering the project, Bugs/Issues are handled on time with respect to their priorities

c. Cost of the project:

- In MNC companies project costs are counted on an hourly basis. Customers have to pay this payment depending upon the resource utilization as well as time complexity.

d. Time Delivery:

- At the time of resources gathering and documentation, the expected build duration of the project gets decided.

e. Escalation:

- If the company exceeds the delivery time then company have to pay the penalty/fine, it is known as escalation.

Who decide the process for software?

1. If client have their own IT department then they will define the process or approach for software.
2. If not then our company will decide the process for software.

Team Size:

- **Project Team-** New features/ functionality/ Module. Ex. Paytm – Invest in stock.
    a. Project team size is 24 to 26 people.
        I. **Deliver manager (1) –** Project delivery to client.
        II. **Project manager (1) -** Project task assign/ work need perform with the time/ team handling.
        III. **Business analysis (1) -** BA interaction client and collect the requirement/ functionality/ New features.
        IV. **Designer/ Solution architecture (1) -** project application design.
        V. **Developer (14 to 16) –** Developer will code for application.
        VI. **Tester (5 to 6) –** Tester will do testing on developer application.

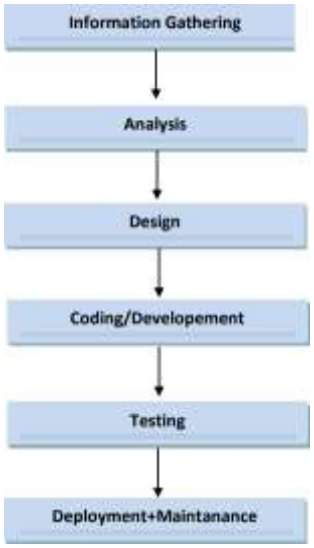
- **Support Team –** Existing application issue/ defects/ end user quires/ Ticket Ex. Paytm-Recharge module
    I. Support team size 9 peoples.
    II. **Project manager/ Support manager (1) -** Support task assign/ work need perform with the time/ support team handling.
    III. **Developer (5 to 6) –** Developer will code for application.
    IV. **Tester (1 to 2) –** Tester will do testing on developer application.

Types of Process Model for building the software?

1. Basic SDLC
2. Waterfall Model
3. V-Model
4. Agile Methodology or model

❖ Basic SDLC or Software Development Life Cycle:

1. This is the generic process for developing any software, Also it is the systematic approach to build the software as per the customer's requirement.
2. **Phases of SDLC:**



3. SDLC process involves development & testing of the application.
4. **SDLC stage involved.**
   a) Information gathering
   b) Analysis
   c) Design
   d) Coding
   e) Testing
   f) Support/ Maintenance

- **Information gathering:**

  - Information gathering will be done by BA.
  - BA will collect the requirement from Client related to business of Client.
  - In Information gathering stage BA will prepared a BRS documents.
  - BRS – business requirements specification.
  - BRS defines clients business related requirements.
  - BRS documents will not get to tester.

- **Analysis:**

  - In Analysis stage BA will work.
  - In Analysis stage BA will collect the requirement from client.
  - BA will collect requirements related the functionality of application/ software.
  - In Analysis stage BA will prepared SRS (Software requirements specification).
  - SRS defines as functional requirements that will be implemented & system requirement that will be used.
  - SRS also called **FRS (functional requirements specification)/ CRS (Customer requirements specification).**
  - SRS will contains:
    - Functional requirements
    - Functional flow diagram
    - Use cases (1 specific requirement)

- Description – Detail about the specific requirement.
- Acceptance criteria – Does & don't about specific requirement/ Summary.

♦ Screenshot/ Snapshot/ prototype

- After completion of SRS documents BA will sent a mail these SRS documents to developer &tester.
- Developer & tester will analysis/ understand/ study the SRS documents.
- If documents is not clear then we will do the meeting with BA.
- **Ex. Use cases-Zerodha Kite- Stock Module- Intraday buy & Sell form new window.**

a) **Description-** User has to buy or sell throw intraday window. For end user intraday added which can buy or sell share / stock. In intraday, user can hold stock/share within 1 day. I.e. within trading time as per your country.

b) Acceptance Criteria:

1.	Intraday available in Stock module for all user.
2.	Intraday for active with trading day (trade cycle time – 9.15 to 3.15 pm).
3.	Throw Intraday user can 1st buy then sell OR 1st sell then buy.
4.	Intraday will provide margin for all stock/share.
5.	Intraday will provide margin for all future & options.
6.	After trade time, for Intraday error message will show – "Intraday not available you order will place after next trading day".

Design:

- In design stage designer or solution architecture will work.
- Designer will prepared the design same as SRS documents.
- Designer will prepared the HLD (high level design), LLD (low level design).

Coding:

- In Coding stage developer will work.
- Developer will work on LLD (low level design).
- LLD will implemented according to SRS/ as per Use cases testing.

Testing:

- Tester will do the test cases design (TCD) & test case execution (TCE).
- Tester will check functionality as per SRS documents OR as per use cases.
- In TCE, if we found a defect/ bug, tester will raised to developer.
- Developer will fix these defect/ bug.
- At the end application/ software deploy/ delivery to the client.
- Project team will give the warrant support for 1 month.

- **Support/ Maintenance:**
  - After completion of project team warrant time/ period then project/ application went to support team.
  - Existing application issue/ defects/ end user quires/ Ticket.

Quality Assurance: Activities focused on providing confidence that quality requirements will be fulfilled.

Quality control: involves various activities, including test activities that support the achievement of appropriate levels of quality.

Software Configuration Management:

- Involves various activities, including test activities that support the achievement of appropriate levels of quality.
- It is a methodology to control and manage a software development project.

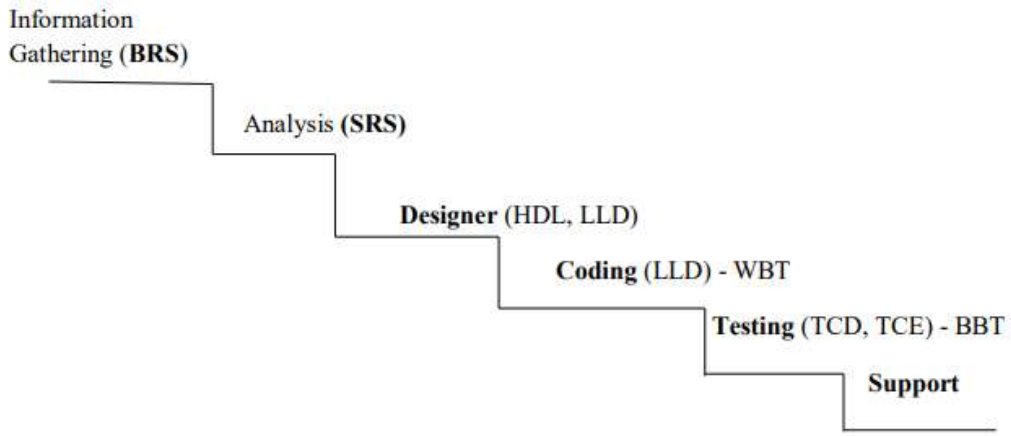
Explain Waterfall Model in Detail?

Waterfall Model:

a) Waterfall model/ process it is also sequential process.
b) Sequential process after completion of one next stage will start with next stage.
c) Ex. Until Developer will complete, tester will not start test testing.

Disadvantages of Waterfall Model:

- Requirement changes are not allowed.
- If there is defect in Requirement that will be continued in later phases.
- Total investment is more, because time taking for rework on defect is time consuming which leads to high investment.
- Testing is start after coding so we have less time available to check the build.

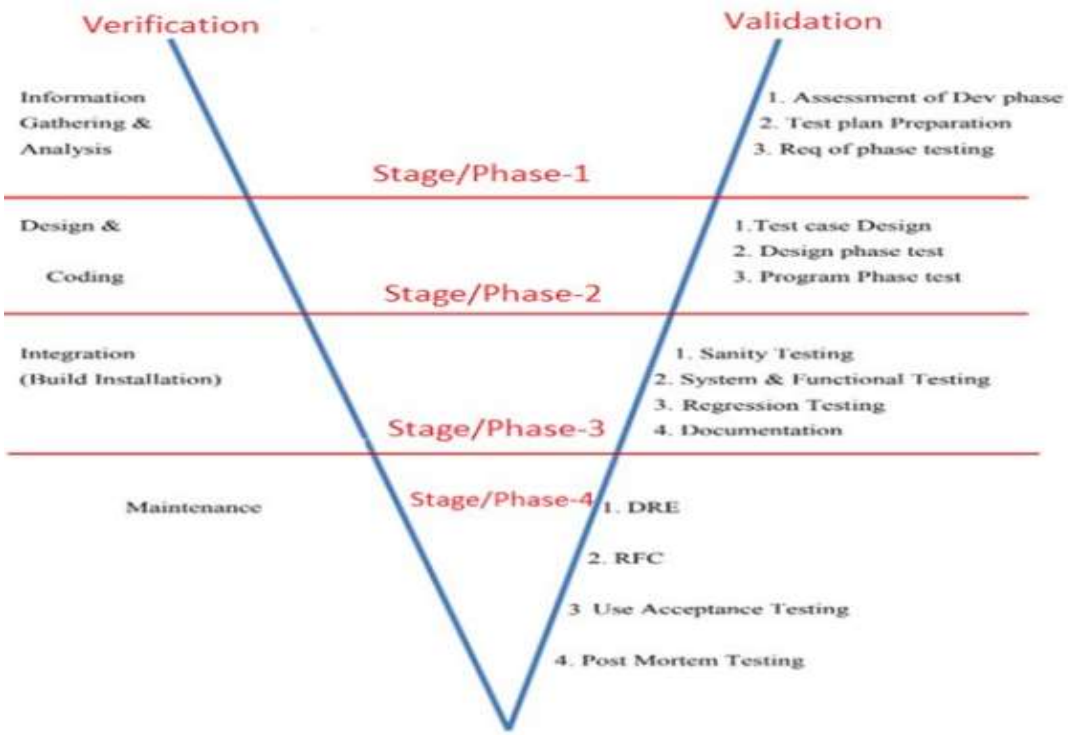


Advantages of Waterfall Model:

- Since Requirement changes are not allowed, chances of finding bugs will be less.
- Initial investment is less since the testers are hired at the later stages.
- Preferred for small projects where requirements are fizzed.

Explain V-Module in Detail?

V-Module:



1. V-module stand for *Verification & Validation*.
2. V-module / process Development (LCD) and testing (LCT) are doing parallel.
3. **Development & testing** stages are work parallel.
4. In V-module / process deployment process is around 3 month.
5. V- Module it is **plan driven process** (deployment process = 3 month)
6. We can accept **change request** at any stage.
7. In verification or Life Cycle development the phases are same as in Software Testing Life Cycle.
8. But these stages are mapped with testing stages known as Life Cycle Testing or Validation.

- **Life Cycle Testing:**

  - **In phase 1,**

    a) Assessment of development phase:
    - In this step PM or TL involved.
    - They decide strategy &Methodology for the testing.
    - Methodology is nothing but which type of testing they preferred like manual, automation, API,Database etc.
    - Strategy is nothing but which type of programming language they preferred like java, ruby,python, Junit, Nunit etc.
    - They also take care of TRM it is nothing but test responsibility matrix.
    - TRM is the mapping of work & Resources.

    b) Test Plan Preparation:
    - In this phase, PM and Test lead is involved.
    - In Test plan preparation, Project Manager and Test lead is responsible for implementation of TRM.
    - They prepare test team & distribute the work to all team member.

    c) Requirement of phase Testing:
    - In this phase QA lead is involved
    - He decides how much test cycles are required and also decide time required for each test cycle.

  - **In phase 2,**

    d) Test Case Design:
    - In this phase QA are involved.
    - They prepared test cases & find out scenarios.
    - According to the requirements they consider positive as well as negative scenarios.
    - Test cases can be maintain with the help of excel based document or special tool like Test link.

    e) Design Phase Testing:
    - In this phase, Architecture & UI developer involved.
    - This testing ensure that build design is correct or not with respect to requirements.

    f) Program Phase Testing:
    - In this phase, front end & back end developer involved.
    - This testing is also known as unit level or code testing.
    - From this testing they ensure that code is working as per expected or not.

  - **In phase 3,**

    g) Sanity Testing:
    - In this testing QAs are involved.
    - Here we check basic core functionality or build stability of the application.
    - If any critical issue or blocker then only we communicate with the developer, minor issue are not considered in sanity testing.

h) System and Functional Testing (SIT):

- Here we execute test cases as per the steps written in test case sheet documents.
- We perform functional & non-functional testing.
- For each fail test case, we logged an issue.
- InSIT, from small issue to big issue all are been logged and communicate with the developer.

i) Regression Testing:

- In this testing QAs are involved.
- Here we check the impact of **newly added module with existing module**.
- In this testing, here we check **only positive scenario's** if time permits negative scenarios as well.

j) Documentation:

- Test documents are created by the individual testers.
- From this document we ensure that who did the testing on particular module & what was status of test case execution.
- After all individual documentation, Test lead create a document which is known as **Test summary report**, which contains all the test case execution records.

- **In phase 4,**

k) Defect removal efficiency (DRE):

- It is a process which calculates at which level tester did the testing, while in BBT, regression and UAT.
- **Formula of Calculate DRE:**
- ***[defect found in SIT / (defect found in SIT + defect found in UAT)]*100***
- Ideal value of defect removal efficiency is equal to one.

l) RFC (Request for Change):

- If any change request come before the product release or during development of product we can easily accept that point should be highlight in BRS document.
- As per the requirement client have to pay extra amount for that.

m) UAT (User Acceptance Testing):

- After removing all open & logged defect found in SIT then product will move to UAT environment to perform UAT testing.
- In UAT, here we check ***end to end Functional flow of application***.
  - From UAT we ensure that the application is defect free & ready for production.

n) Post Mortem Testing:

- If any unexpected behavior of the system before the product release at that time we communicate with the developer find the root cause and fix that issue at the same time.

- **In the Life Cycle Development:**
  - **Phase 1:**
    - **Information Gathering and Analysis:**
      - BA collects the requirements from the client and create business requirement specification document.
      - BA converts business requirement specification document into the software requirement specification and send them to the team.
      - SRS documents will contains the functional requirements, functional flow diagram, use cases and prototype.
  - **Phase 2 :**
    - **Design and coding:**
      - In design stage designer or solution architecture will work.
      - Designer will prepared the design same as SRS documents.
      - Designer will prepared the HLD (high level design), LLD (low level design).
      - In Coding stage developer will work.
      - Developer will work on LLD (low level design).
      - LLD will implemented according to SRS/ as per Use cases testing.
  - **Phase 3:**
    - **Integration:**
      - It is performed by developer.
      - They merge or integrate new code with existing build of the application.
      - Once white box testing is done by developer, then they will add new code with existing build.
      - After successful integration they perform integration testing from this they ensure that code is successfully merge or not.
  - **Phase 4:**
    - **Support/Maintenance:**
      - After deliver the product on client side, we need to take care the issue or defect are logged by client.
      - We have to handle that issue based on the priority of client.

What is Static Testing?

1. Testing the project related documents is called *as **static testing.***
2. It is also known *as **verification***.
3. Verification checks whether we are building the right product.
4. This typically involves review, walkthrough, inspection.
5. Static testing / verification we will do/ check ***quality assurance.***
6. Static testing involves ***review, walkthrough, inspection*** etc.
7. **Review:**

a) **Requirement reviews:** business analyst will check their documents.
b) **Design reviews:** designer will check their design whether it is according to the requirements.
c) **Code reviews:** developer will check their code or logic.
d) **Test plan reviews:** Test lead will check the test plan.
e) **Test cases reviews:** The individual tester will check their test cases whether he covers all the functionality of the application or not.

8. Walkthrough:

a) It is informal review.
b) Author reads the documents or code and discuss with peers.
c) It's not pre -planned and can be done whenever required.
d) Also, walkthrough does not have minutes of the meet.
e) At least 2 people are required to perform the review.

9. Inspection:

a) It's a most formal review type.
b) In which at least 3- 8 people will sit in the meeting 1- reader 2-writer 3- moderator plus concerned.
c) Inspection will have a proper schedule which will be intimated via email to the concerned developer/tester.

What is Dynamic Testing?

a) It is also known *as Validation.*
b) Validation checks whether we are building the ***product right.***
c) Dynamic testing involves the actual testing of the software.
d) Dynamic testing takes ***place after verifications*** are completed.
e) Dynamic testing / validation we will do/ *check* ***quality control.***
f) It focuses on the software.
g) Dynamic testing involves testing such as unit testing, integration testing, system and functional testing and user acceptance testing.

Advantages:

- Testing is involved in each and every phase.

Disadvantages:

- Documentation is more. Initial investment is more.

Difference between Quality Assurance and Quality Control?

A Quality Assurance:

1. QA focuses on building in quality.
2. QA is preventing defects.
3. QA is process oriented
4. QA for entire lifecycle.
5. It is nothing but the verification.

B. Quality Control:

1. QC is the actual testing of the software.
2. QC focuses on testing for quality.
3. QC is detecting defects.
4. QC is product oriented.
5. QC for testing part in SDLC.
6. It is nothing the validation

Agile Methodology:

1. It is an iterative and incremental approach.
2. Agile methodology follows iterative and incremental process.
3. **Agile Process:**
4. Customers do not have to wait for long time.
5. We can develop, test and deploy the piece of software to the stakeholder with few features.
6. We can accept the requirement changes.
7. There is a good communication between product owner, stakeholder, and scrum master and development team.
8. Agile defines continuous development & continuous testing will happen in application/ build.
9. Agile process it is value driven process.
10. If any change request comes at any point of time then we will consider these change request.
11. When will accept these CR then we will check impact on current development, testing & production.
12. If change request has more impact on development/ testing/ production then we will inform to client.
13. If change request has less impact on development/ testing/ production then we will develop & testing and we will deploy to client.
14. In Agile process deployment process in 2 week.
15. Agile subtypes/ framework/ methodology
    a) **XP (extreme programming)** – Development & no testing.
    b) **Scrum** - (Bunch of requirement then Sprint wise development & Testing & Delivery).
    c) **Kanban** – Support team (tickets/ Existing issue/ bugs/CR).
    d) **Lean** - Support team.
    e) **FDD** – Future driven development.
16. I have worked in Scrum agile methodology.

Advantages of Agile Methodology:

a) Requirement changes are allowed at any stage of development.
b) Release will be very fast.
c) Customers no need to wait for long time.
d) Good communication between team.
e) It is an easy model to adopt.

f) Automation is possible is in agile process.
g) Checkpoints are added after each module.
h) Agile ceremonies is better advantage by using this we can track the project

Disadvantage of Agile Methodology:

a) If frequent changes in requirement/ US, we can't deployment/ delivery these US to client.
b) If your application/ module is depend on another application/ module, we can't deployment/ delivery these US to client.
c) Less focus on the design and documentation since we deliver the software very faster.

Terminologies used in agile:

a) **Agile:** It is continuous process of development & testing.
b) **Sprint:** It is collection of requirement or bunch of user stories.
c) **Burn down Chart:** How much user story remaining with respect to time. In simple words, how much work remaining with respect to time.
d) **Burn up Chart:** How much user story completed with respect to time.
e) **Estimation:** Time slot provide to complete the work or User Story.
f) **Velocity:** Amount of work completed within sprint with respect to the estimation. It define sprint wise how much deployment of user stories to client.
g) **Epic:** It is nothing but the Main Module.

Agile Ceremonies:

a. Grooming Session:

- **Host:** Scrum Master
- **Involvement:** scrum master, product owner, development team.
- **Happens when:** before the sprint planning meeting.
- **Duration:** 1-1.5hrs
- **Discussion:** Here PO explains about the all user stories to team, those user stories which aregetting shortlisted for the sprint.

b. Sprint Planning Meeting:

- **Host:** Scrum Master
- **Involvement:** scrum master, product owner, development team.
- **Happens when:** before the sprint start.
- **Duration:** 2-2.5hrs
- **Discussion:** User Stories shortlisted here. Discussion about the user stories. Everyone share thought about the time, approach, strategy for development and testing. Based on this story points gets decided to each user stories.

c. **Daily Standup Meeting:**

- **Host:** Scrum Master
- **Involvement:** scrum master, product owner, development team.
- **Happens when:** Everyday
- **Duration:** 15-20min
- **Discussion:** Here everyone provide their work update to the scrum master. If any blocker in work then communicate with the team and get the solution.

d. **Sprint Review Meeting:**

- **Host:** Scrum Master
- **Involvement:** scrum master, stakeholder, product owner, development team.
- **Happens when:** at the sprint end.
- **Duration:** 30-60 min
- **Discussion:** Here we have to provide demo on what we completed within the sprint to the stakeholder. Then Stakeholder provide their review to the team .This is the time for team celebration for their accomplishment.

e. **Sprint Retrospective Meeting:**

- **Host:** Scrum Master
- **Involvement:** scrum master, product owner, development team.
- **Happens when:** at the sprint end.
- **Duration:** 60 min
- **Discussion:** Here we discuss about the previous sprint. What good things we have done .What bad things we have done .Here we set the goal for next sprint.

What is done & ready Status?

a) **Done:** user story is totally work completed for development & testing. User story deployed to client then Userstory status is **done.**

b) **Ready:** : user story is totally work completed for development & testing .US is not yet deployed toclient then User story status is Ready.

Agile Architecture:

a. **Stakeholder:**

- They are top most body in the company.
- They have all the requirement related to the project.
- They can ask for change request at any phase of development.

b. **Project Owner:**

- Collect all the information from stakeholder.
- Project owner create product backlog based on the requirements share by the stakeholder.
- They select user stories according to the client priority.

c. Sprint Backlog:

- It is prepared by the product owner.
- In Analysis stage product owner will prepared sprint backlog.
- Sprint backlog defies as functional requirements that will be implemented & system requirement that will be used.
- Sprint Backlog will contains:
  - Functional requirements
  - Functional flow diagram
  - Use cases (1 specific requirement)
    - Description – Detail about the specific requirement
    - Acceptance criteria – Does & don't about specific requirement/ Summary
  - Screenshot/ Snapshot/ prototype.
- After completion of Sprint Backlog documents product owner will sent a mail these documents to developer & tester.
- Developer & tester will analysis/ understand/ study the sprint backlog documents.
- If documents is not clear then we will do the meeting with product owner.

d. Design:

- In design stage designer or solution architecture will work.
- Designer will prepared the design same as SRS documents.
- Designer will prepared the HLD (high level design), LLD (low level design).

e. Estimation:

- In Estimation project owner, development lead, test lead is involved.
- Estimation is having 3 factors: (knowledge, efforts, Complexity)
  - **Knowledge:**
    - Whenever team formation is done, each team member should have knowledge about domain of the product.
  - **Efforts:**
    - High level management decides how much efforts & resources are required. As per dependencyof module. They select user story.
  - **Complexity:**
    - This is the important factor in Estimation for defining complexity.
    - They consider time, cost, resources requirements, Bandwidth of team.

f. Coding:

- In Coding stage developer will work.
- Developer will work on low level design.
- Low level design will implemented according to SRS/ as per use cases.

g. Testing:

- Tester will do the test cases design & test case execution.
- Tester will check functionality as per software requirement specifications documents or as per use cases.

- While test case execution, if we found a defect/ bug, tester will raised to developer.
- Developer will fix these defect/ bug.
- At the end application/ software deploy/ delivery to the client

h. Support/ Maintenance:

- After completion of project team warrant time/ period then project/ application went tosupport team existing application issue/ defects/ end user quires/ ticket.
- Project team will give the warrant support for 1 month.

Difference between scrum and Kanban

Scrum	Kanban/lean
Scrum will follow in project team.	Kanban will follow in support team.
Team member will defined work assigned	Team member not defined work assigned.
In scrum will work on sprint 2 week.	Kanban will work on CR
Sprint deployment in 2 week.	In Kanban CR & defects a work deployment 2-3 weeks
In scrum priority of work already decided	In Kanban priority of work not decided.

Scrum:

- Scrum is a framework through which we build the software product by following agile principles.
- Scrum normally includes the group of people called as scrum team.
- Normally 8-9 peoples in scrum team.
  - Product owner
  - Scrum master
  - Development team
  - QA team

    - **Product owner:**
      - Define the feature of the product.
      - Prioritize features according to the market value.
      - Adjust features and priority of every iteration, as needed.
      - Accept or reject the work results.

    - **Scrum master:**
      - The main role is facilating and driving the agile process.

    - **Development and QA Team:**
      - Develop and test the software.

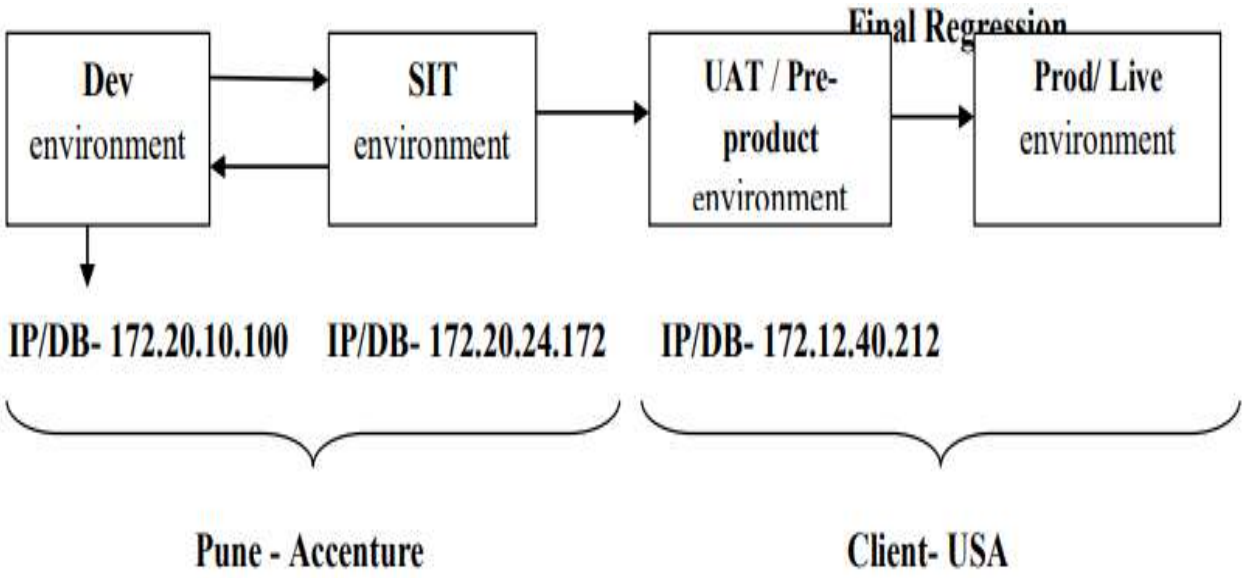
- **Sprint:**
  - It is nothing but the period of time required to complete user story decided by product owner and team. Usually it is 2-4 weeks.

- **Story Points:**
  - It is rough estimation of user story is given by developer and QA in the form of Fibonacci series.

❖Testing Environments:

In my project 4 types environment

- **Dev environment -** developer work +coding + white box testing
- **SIT environment (System integration testing)** – tester test case design, test case execution ,defect and defect report
- **UAT environment -** Client location (Developer + tester)
- **Prod/ Live environment** -End user application



- **Dev environment:**
  - Developer will do coding & white box testing.
  - Developer will sent the build deploy into SIT environment.
  - Dev environment (feature 250 line code) then do build delivery (250 line) to SIT environment (feature-250 line code).
  - Developer will sent Mail to Tester (throw JIRA/ HAPLM)
  - In Mail Attachment – Unit testing documents which contains screenshot for testing white box testing, table's name, URI/URL, etc.

- **SIT environment**:
  - Tester will work.
  - Tester do test case design documents and test case execution accordingly.
  - In TCE if we found defects then we will raised to developer.
  - Inform throw Mail (JIRA/ HAPLM).
  - Defect will fix by developer in dev environment then they sent Modified Build.
  - Tester will check defects
  - In SIT environment while TCE, tester will prepared the test proof then send email to UAT team throw JIRA/ HPAML tool.
    - **Dev environment URL = 172.20.10.100:8080 /https://www.flipkart.com/**
    - **SIT environment URL = 172.20.24.172:8080/ https://www.flipkart.com/**

- **Dev environment Testing:**
  - Unit Testing
  - Integration Testing

- **SIT environment Testing:**
  - Sanity Testing/ Smoke Testing
  - System & functional Testing (BBT)
  - Re-testing
  - Regression Testing, etc.

- **UAT environment Testing:**
  - Alpha Testing
  - Beta Testing

- **Production/live environment testing:**
  - Production Issue

Dev environment:

- In Dev environment, developer will work.
- Developer will do coding as per user story.
- After completion of coding developer will do testing (WBT)
  - Unit Testing
  - Integration Testing

  - **Unit Testing:**
    - A unit is a single component or module of a software.
    - Unit testing conducts on a single program or single module.
    - Unit testing is white box testing technique.
    - Unit testing is conducted by the developers.
    - Unit testing documents will contains screenshot for testing WBT, Tables Name, URI/URL, etc.
    - **Unit testing techniques:**

      1. Basis path testing (Every path is tested).
      2. Control structure testing
      3 Conditional coverage (will check all conditions in program will be verified by true and false)
      4. Loops Coverage
      5. Mutation Testing

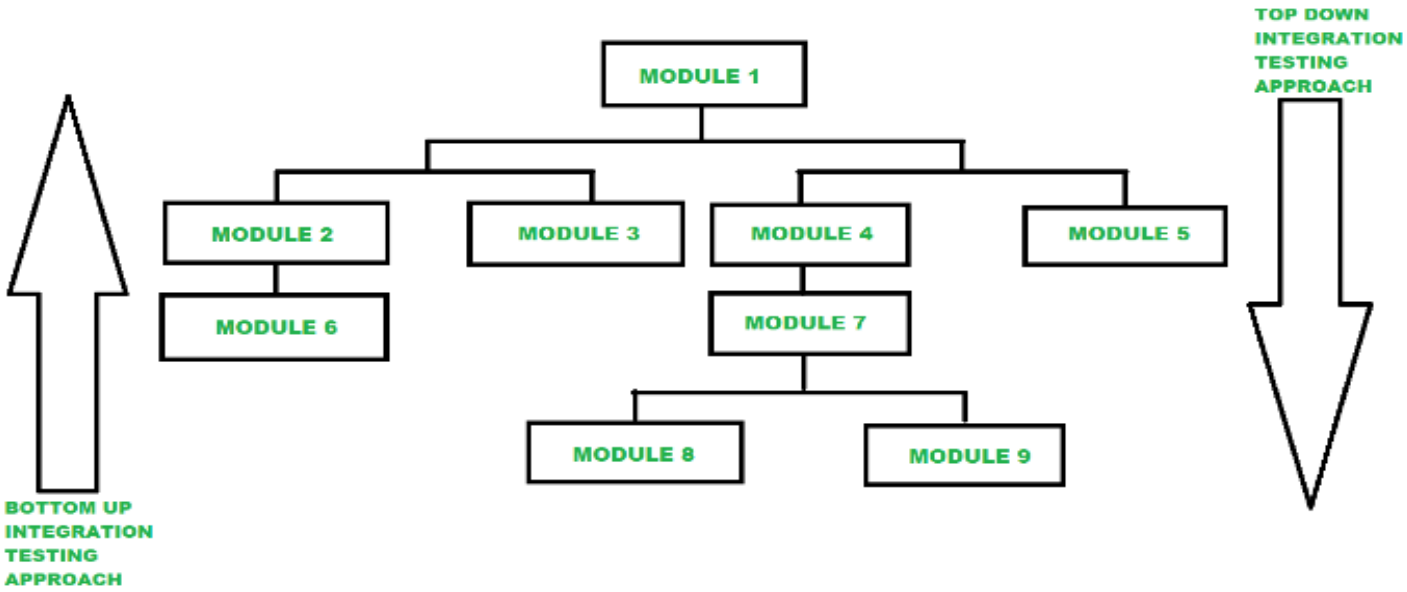
    - **Integration Testing**:
      - Integration testing performed between 2 or more modules.
      - Integration testing focuses on checking data communication between multiple modules.
      - Integrated testing is white box testing technique.
      - **Types of integration testing:**
        - Incremental integration testing
        - Non- Incremental integration testing

        - **Incremental integration testing:**
          - Incrementally adding the modules and testing the data flow between the modules.
          - Incremental Integration Testing is having 3 approaches:
          - **Top Down -Incremental integration Testing:**
            - Incrementally adding the modules and testing the data flow between the modules and ensure module added is the child of previous module.
            - If we don't have sub-module then developer will prepared stub.
            - Stub- It is temporary program prepared by developer throw XML languages

- **Bottom up -Incremental integration Testing:**
  - Incrementally adding the modules and testing the data flow between the modules.
  - And ensure module added is the parent of the previous module.
  - If we don't have main-module then developer will prepared driver.
  - Driver - It is Temporary program prepared by developer throw XML languages.

- **Sandwich or hybrid Approach:**
  - Combination of top-down & bottom-up approach is called as sandwich approach.
  - Here main-module and sub-module is not implemented so in that case developer will have to create driver and stubs to test the functionality of the browser.

- **Non-Incremental integration Testing:**
  - Adding all the modules in a single shot and test the data flow between modules.
  - **Drawbacks of non-incremental integration Testing**:
  - We might miss data flow between some of the modules.
  - If we find any defect, we can't understand the root cause of defect.



SIT environment:

- Tester will work.
- Tester do test case design documents and test case execution accordingly.

- **SIT environment Testing:**
  - Sanity Testing/ Smoke Testing
  - System & functional Testing (BBT)
  - Re-testing
  - Regression Testing, etc.

Sanity Testing:

- Sanity Testing is a **zero-level testing (**also called **Build Acceptance Testing** or **Tester Acceptance Testing**) performed on a new build provided by the developer.
- It ensures that the build is stable enough for further testing.
- In SIT environments, Sanity testing it is a first testing which is performed, sanity testing is called **zero level testing.**
- Developer shares a new build via email or defect tracking tool (e.g., JIRA / HP ALM).
- Conducted in SIT (System Integration Testing) environment.
- Executed immediately after receiving the build.

- **Purpose of Sanity Testing:**
  - To validate the stability of the build.
  - To check whether the build is ready for detailed testing.
  - If the build is not stable → Tester rejects the build and informs the developer (via Mail/JIRA/HP ALM).

- **Validation Performed in Sanity Testing :**

  a. **Core Functionality Validation :**
     - Verify if critical functionalities work as expected as per requirements.
     - Example checks: Login button, Agree/Skip/OK buttons, Test button.

  b. **GUI (Graphical User Interface) Validation :**
     - Verify user interface elements like logo size, page dimensions, images clarity, and quality.

  c. **Links Validation :**
     - Ensure hyperlinks are active and redirect to the correct functionality/module.

  d. **Tabs Validation :**
     - Verify behavior of web elements like text boxes, search bars, and input fields.
     - Example: Enter username/password, search functionality, entering text.

  e. **Page Navigation Validation :**
     - Check page flow and navigation to ensure users can move to the next page/module smoothly.

Effort & Involvement

- Typically requires **2 to 3 hours** per new build.
- Performed **only by the tester**.

Smoke Testing:

- Smoke Testing is an advanced version of Sanity Testing
- It is performed whenever a new build is released to check build stability before moving forward with detailed testing.
- Purpose: To validate that the critical functionalities, UI elements, and navigation flows are working correctly.

- **Validation Performed in Smoke Testing :**

  a. **Core Functionality Validation :**
     - Verify if critical functionalities work as expected as per requirements.
     - Example checks: Login button, Agree/Skip/OK buttons, Test button.

  b. **GUI (Graphical User Interface) Validation :**
     - Verify user interface elements like logo size, page dimensions, images clarity, and quality.

  c. **Links Validation :**
     - Ensure hyperlinks are active and redirect to the correct functionality/module.

  d. **Tabs Validation :**
     - Verify behavior of web elements like text boxes, search bars, and input fields.
     - Example: Enter username/password, search functionality, entering text.

  e. **Page Navigation Validation :**
     - Check page flow and navigation to ensure users can move to the next page/module smoothly.

- **Key Advantage of Smoke Testing:**
  - **Sanity Testing** = Core checks of build stability.
  - **Smoke Testing** = Sanity Testing **+ Troubleshooting** (by tester) or **+ Package Validation** (by developer).

- **Key Notes:**
  - Test cases are **not written** for Smoke Testing.
  - If the build is unstable, it is **rejected immediately**.
  - Issues may arise due to:
    - Environment problems (SIT)
    - System hang/runtime failures
    - Core/basic functionality issues

Difference between smoke and Sanity Testing?

Smoke Testing (Build Verification Test):

a. **Approach:**
   - It is a **broad and shallow testing approach** where all major modules and functionalities of the application are checked at a high level.
   - The goal is not to go into detail, but to make sure the overall system works.

b. **Purpose:**
   - Used to validate the stability of the build. If the smoke test fails, the build is rejected immediately and sent back to the development team

c. **Scope :**
   - Focuses on **critical and major functionalities** like login, navigation, and core workflow.

d. **Subset :**
   - Considered as a **subset of acceptance testing** because it ensures the build is stable enough to proceed with further testing.

e. **Execution:**
   - Can be done **manually or automated**, depending on the project requirements.

f. **Performed By:**
   - Both **developers and testers**. Developers may run smoke tests before releasing the build, and testers validate again in QA

g. **Depth :**
   - Testing is done without getting into deep details, but if needed, testers may drill down to confirm critical failures

h. **Other Name :**
   - Often called **Build Verification Testing (BVT)** because it decides whether the build is testable.

i. **When :**
   - Performed on every new build release.

Sanity Testing:

a. **Approach:**
   - It is a narrow and deep testing approach where only specific functionalities of the application are tested

b. **Purpose:**
   - The goal is to validate rationality and correctness of specific functionalities after minor changes, bug fixes, or small enhancements

c. **Scope :**

- Limited to only those areas or modules where code has been modified. For example: if login functionality was fixed, sanity testing will focus on login-related scenarios only.

d. **Subset :**

- Considered as a **subset of regression testing**, since it ensures that new fixes/changes do not break the related functionality.

e. **Execution:**

- Usually done **manually without detailed test cases or scripts**. Testers rely on their understanding of requirements and recent changes.

f. **Performed By:**

- Only by **testers** in the QA environment. Developers are not usually involved

g. **Depth :**

- Unlike smoke testing, sanity testing goes deeper into specific functionality, but does not cover the entire system

h. **When :**

- Performed after receiving a stable build but with small modifications or fixes to confirm they work as expected.

b. **Outcome :**

- If sanity testing fails, the build is **rejected** and developers are informed to re-fix the issues.

Types of Black Box Testing:

a. **Functional testing :**
- Functional testing is a type of software testing in which the system is tested **against the functional requirements and specifications**.
- Functional testing ensures that the requirements or specifications are properly satisfied by the application.

b. **Non-Functional testing :**

- Non-functional testing is a type of software testing that is performed to verify **whether the behaviour of the system is as per the requirement or not.**

Difference between Functional and Non-Functional Testing:

Functional Testing:
- It verifies the **operations, features, and actions** of an application.
- It is based on **business requirements** and validates *what the system should do.*

- **Examples: Unit Testing, Integration Testing, Regression Testing, Sanity Testing, Smoke Testing, User Acceptance Testing (UAT).**

Non-Functional Testing:

- It verifies the **behavior, performance, and quality attributes** of an application.
- It is based on performance and non-functional requirements, ensuring how the system performs.
- **Examples: Performance Testing, Load Testing, Stress Testing, Security Testing, Usability Testing, Compatibility Testing.**

Types of Functional Testing:

a. Unit testing :

- Here, individual unit/ component of an application are tested. It is done during the development phase.
- For e.g., Like If Developer created any website login page and developer will log in with a correct password and a correct username and it will redirect the user to result in the page it is called successful unit testing

b. Integration testing :

- Here, individual unit/component are combined or, grouped together and is tested.
- For e.g., in Gmail if we send mail to anyone and verify in the Send Items folder to check if the send mail is there.

c. System testing :

- Here, complete integrated software is tested (End-to-End) to verify the customer requirement.
- Application as a whole is tested using functional and non- functional testing.
- It is carried out by the QA team after the completion of integration testing and before the final acceptance testing.

d. User-acceptance testing :

- It is a type of testing performed by the end user or the client to verify/accept the software system before moving the software application to the production environment.
- UAT is done in the final phase of testing after functional, integration and system testing is done.

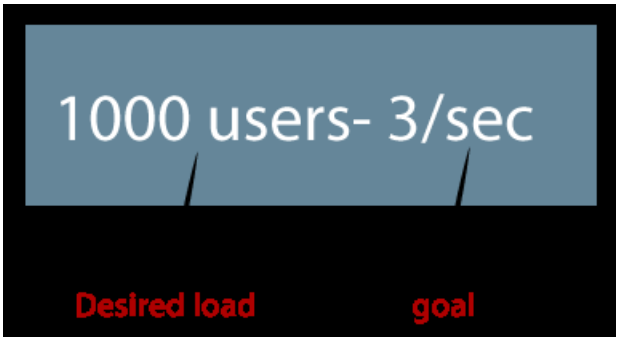
Type of Non-Functional Testing:

1. Performance Testing:

- Performance Testing eliminates the reason behind the slow and limited performance of the software.
- Reading speed of the software should be as fast as possible.

- For Performance Testing, a well-structured and clear specification about expected speed must be defined. Otherwise, the outcome of the test (Success or Failure) will not be obvious.
- While doing performance testing on the application, we will concentrate on the various factors like **Response time, Load, and Stability** of the application.
- **Response time:** Response time is the time taken by the server to respond to the client's request.
- **Load:** Here, Load means that when N-number of users using the application simultaneously or sending the request to the server at a time.
- **Stability**: For the stability factor, we can say that, when N-number of users using the application simultaneously for a particular time.

- **Load Testing :**
    - Load testing involves testing the system's loading capacity.
    - Loading capacity means more and more people can work on the system simultaneously.
    - The load testing is used to check the performance of an application by applying some load which is either less than or equal to the desired load is known as load testing.
    - For example: In the below image, 1000 users are the desired load, which is given by the customer, and 3/second is the goal which we want to achieve while performing a load testing



- **Stress Testing :**
    - The stress testing is testing, which checks the behaviour of an application by applying load greater than the desired load.
    - For example: If we took the above example and increased the desired load 1000 to 1100 users, and the goal is 4/second. While performing the stress testing in this scenario, it will pass because the load is greater (100 up) than the actual desired load.

- **Scalability Testing:**
  - Checking the performance of an application by increasing or decreasing the load in particular scales (number of users) is known as **scalability** testing.
  - Upward scalability and downward scalability testing are called scalability testing.
  - Scalability testing is divided into two parts which are as follows:
  - **Upward scalability testing:** It is testing where we increase the number of users on a particular scale until we get a crash point. We will use upward scalability testing to find the maximum capacity of an application.
  - **Downward scalability testing:** The downward scalability testing is used when the load testing is not passed, then start decreasing the no. of users in a particular interval until the goal is achieved. So that it is easy to identify the bottleneck (bug).

2. Security Testing :

- Security testing is used to detect the security flaws of the software application.
- The testing is done via investigating system architecture and the mind set of an attacker.
- Test cases are conducted by finding areas of code where an attack is most likely to happen.

3. Portability Testing :

- The portability testing of the software is used to verify whether the system can run on different operating systems without occurring any bug.
- This test also tests the working of software when there is a same operating system but different hardware.

4. Reliability Testing :

- Reliability test assumes that whether the software system is running without fail under specified conditions or not.
- The system must be run for a specific time and number of processes.
- If the system is failed under these specified conditions, reliability test will be failed.

5. Efficiency Testing

- Efficiency test examines the number of resources needed to develop a software system, and how many of these were used. It also includes the test of these three points.

- Customer's requirements must be satisfied by the software system.
- A software system should achieve customer specifications.
- Enough efforts should be made to develop a software system.

Advantages of Non-functional testing:

- It provides a higher level of security. Security is a fundamental feature due to which system is protected from cyber-attacks.
- It ensures the loading capability of the system so that any number of users can use it simultaneously.
- It improves the performance of the system.
- Test cases are never changed so do not need to write them more than once.
- Overall time consumption is less as compared to other testing processes.

Disadvantages of Non-Functional Testing:

- Every time the software is updated, non-functional tests are performed again.
- Due to software updates, people have to pay to re-examine the software; thus software becomes very expensive

System & functional Testing (BBT):

- Testing over all functionality of the application with respective client requirements.
- It is a black box testing technique.
- This testing is conducted by testing team.
- After completion of component and integration level testing's we start system testing.
- Before conducting system testing, we should know the customer requirements.
- *System testing focusses on below aspects.*
  - **Usability testing**
  - **Functional testing**
  - **Non-Functional testing**
  - **Performance Testing**
  - **Security testing**
  - **User interface testing (GUI):**
    - Graphical User-interface Testing is a process of testing the user interface of an application.
    - A graphical user interface includes all the elements such as menus, checkbox, buttons, colors, fonts, sizes, icons, content, and images.
    - **GUI Testing Checklist**
      - Testing the size, position, width, height of the elements.
      - Testing of the error messages that are getting displayed.
      - Testing of the font whether itis readable or not.
      - Testing of the screen in different resolutions with the help of zooming in and zooming out.

- Testing the alignment of the texts and other elements like icons, buttons, etc. are in proper place or not.
- Testing the colors of the fonts.
- Testing whether the image has good clarity or not.
- Testing the alignment of the images.
- Testing of the spelling.
- The user must not get frustrated while using the system interface.
- Testing whether the interface is attractive or not.
- Testing of the scrollbars according to the size of the page if any.
- Testing of the disabled fields if any.
- Testing of the size of the images.
- Testing of the headings whether it is properly aligned or not.
- Testing of the color of the hyperlink.
- Testing UI Elements like button, textbox, text area, check box, radio buttons, drop downs links etc.

- **Usability testing:**
  - Usability testing, tester we will validate user friendlessness of the screen/ application.
  - **Usability testing 2 type:**
  - **GUI(graphical user interface) / UI(user interface) Testing:**
  - **In GUI/ UI testing, as tester validation**.
    - Validation look & feel of the screen / application.
    - Validation ease to use (End user ease) of screen / application.
    - Speed of interface.
  - **Manual support Testing-**
    - In manual support testing, as tester we will validate, how manual text will be support in screen / sensitiveness on screen.

- **Functional testing:**
  - Functionality is nothing but behavior of application.
  - Functional testing talks about how your features should work.
  - In Functionality, tester will validate internal feature of the application/ build.
  - For validating internal feature we will performed different coverage.
  - **Functional testing having testing converges:**
    - **Behavioral coverage testing-**
      - Tester validates the behavior of objects/web elements present in the application.
      - It includes checking how the object/web element responds when interacted with (e.g., buttons, links, text boxes, and dropdowns).

- Tester also verifies the properties and expected behavior of the object/web elements as per the functional requirements

Object/ Web elements	Behavioral & Property
Text box	Focus & Un- focus
Check box	Check & un-check OR Tick & Un-tick
Button	Enables & Disables
Radio button	On & off

- **Input domain converge testing:**
  - In input domain coverage testing, the tester validates the system's behavior with different input values, sizes/lengths, and data types.
  - The focus is on ensuring that the application handles all possible valid and invalid inputs correctly.
  - For this testing, testers maintain and verify various input sizes, lengths, and data types as part of the test design.
  - **Input domain coverage has 3 techniques:**
  - **Boundary value analysis, equivalent class partitioning, decision table technique.**

a. Boundary value analysis:

- BVA is a black-box functional testing technique.
- BVA we will check input size / length.
- In BVA, the tester checks the application's behavior at the boundary values of input ranges, since most defects are found at the edges rather than in the middle of input ranges.
- Instead of testing all possible inputs, the focus is on the minimum, just above minimum, maximum, just below maximum, and typical middle values.
- It helps to reduce the number of test cases while still ensuring effective test coverage.
- Commonly applied in fields such as forms, login screens, numeric ranges, data length fields, etc.
- Example:
  - **Login Page:**
  - Username (Mobile Number): Should accept exactly 10 digits.
  - Test inputs: 9 digits (below boundary), 10 digits (valid boundary), 11 digits (above boundary).

- Password: Must be between 4 to 6 characters with a combination of uppercase, lowercase, and numbers.
    - **Test inputs:**
      3 characters → invalid (below minimum).
      4 characters → valid (minimum boundary).
      5 characters → valid (within range).
      6 characters → valid (maximum boundary).
      7 characters → invalid (above maximum).

b. **equivalent class partitioning:**

- Equivalent Class Partitioning is a functional testing technique where input data is divided into valid and invalid equivalence classes.
- Each class represents a set of inputs that should be treated the same by the system.
- Testing one representative value from each class is considered sufficient.
- **Purpose**: To reduce the total number of test cases while still effectively covering all possible input scenarios.
- **Ex. Login page**
- **Username: Must be a valid mobile number.**
    - **Valid class:** 10-digit numbers.
    - **Invalid class:** Less than 10 digits, more than 10 digits, alphabetic input.
- **Password: 4–6 characters, combination of uppercase, lowercase, and numbers.**
    - **Valid class:** "Ab12", "Xy34Z".
    - **Invalid class: "abc", "1234567", "ABCDEF".**

c. **Decision table technique:**

- Decision Table Testing is a functional testing technique used to validate different combinations of inputs and their corresponding outputs or actions.
- It is particularly useful when multiple conditions or business rules affect the behavior of the system.
- Ex. Ex. Login page- Username- only Mobile no, Password- 4 to 6 charter combination (Capital,Small letters & 2 No.), Submit Objects Rule 1/ input combination 1.

Objects	Rule 1/ input combination 1	Rule 2/ input combination 2	Rule 3/ input combination
Username	Valid	Valid	In-Valid
Password	Valid	In- Valid	Valid
Submit	Press	Press	Press
Result	Home	Error	Error

- **Error handling converge testing:**
  - Verify that the application correctly handles invalid inputs and informs the user appropriately.
  - **Approach:**
    - Enter invalid or incorrect data into input fields or web elements.
    - Perform actions to trigger system validation.
  - **Checks**:
    - Error messages must be clear and readable.
    - Messages should use simple and understandable language for end users.
    - Messages should indicate what exactly went wrong to guide the user.
  - **Examples**:
    - Generic: Incorrect data → not helpful, unclear.
    - Specific: Invalid Username / Password → Informative and actionable.
- **Backend coverage testing/ database converge testing :**
  - Validate that front-end operations correctly store and manipulate data in the backend database.
  - Also known as Database Testing.
  - **Key Focus Areas:**
    - DML Operations: Ensure INSERT, UPDATE, DELETE, SELECT operations work as expected.
    - Table & Column Validation: Verify column types, lengths, number of columns, and overall table structure.
    - Data Relationships: Check relationships between tables for proper normalization and referential integrity.
    - Database Objects: Validate functions, stored procedures, triggers, indexes, and views.
- **Calculation/Manipulation Testing:**
  - *Verify arithmetic calculation with respect to the user perform actions.*
  - Tester should verify the calculations
  - e.g. – Banking application – amount credited and debited should reflect in account.

- **Service based converge testing:**
  - Validating sequential operation/ end to end flow of the application.

- **Links Existence & Links Execution:**
  - **Links Existence:**
    - Verify that all expected links are present on the page.
    - Ensure links are placed correctly according to the design or requirements.

  - **Links Execution:**
    - Verify that each link navigates to the correct destination page.
    - Check all types of links:
      a) **Internal Links:** Navigate within the same website.
      b) **External Links:** Navigate to a different website.
      c) **Broken link:** Ensure no link is broken or leads to an error page

- **Intersystem coverage testing-:**
  - In Intersystem Coverage Testing, we verify how well our application exchanges or shares data with another system or external application.
  - Example: Checking if a healthcare app correctly sends prescription data to an e-pharmacy system, or if a booking app shares appointment data with a payment gateway.

- **Sanitation / Garbage Testing :**
  - In **Sanitation (or Garbage) Testing**, testers verify that the application **does not contain any extra or unwanted features, fields, buttons, or text** that are **not part of the approved requirements or design**.
  - The goal is to ensure that the final product strictly follows the **scope and specifications** provided by the client.
  - **Process:**
    1. During testing, if any **extra functionality or element** is found (e.g., an unused button, field, or hidden feature), it is reported.
    2. The QA team discusses it with the client or product owner.
    3. If the client agrees to keep the extra feature, it can remain. If not, the development team must remove it.
  - **Example:**
    i. Requirement: "The form should have Name and Email fields."
    ii. Developer adds an extra "Phone Number" field without client approval.
    iii. This is considered garbage functionality and should be removed unless approved.

Non-Functional testing:

- In Non-Functionality, tester will validation External feature of the application/ build.
- Non- Functionality testing, different testing types:

Recovery coverage testing:

- In recovery coverage we ensure that whether the system is able to recover from abnormalsituation to normal situation.
- **Ex. Filpkart- Payment page/ tab-while doing payment close or refresh the page -close application-add to card page/ tab.**

Compatibility coverage testing (Browser Compatibility testing):

- In this we check whether the build/application is compatible with the user expected platform or not.
- Here we deal with the User Expected Platforms such as operating system and browser.
- ***Forward Compatibility:***
  - Here check the application compatibility for **newer version of browser or operating system.**
  - If we have problem in operating system of SIT environments then Network team handle that issue not tester.
- ***Backward Compatibility:***
  - Here we check the application compatibility **for older versions of browser or operating system.**
  - *Backward Compatibility testing having two types:*
  - In my project I have done "Browser Compatibility testing".
    - ***Cross browsing Compatibility testing:***
      - Validating application/ build it support different browser.
      - Ex. Different browser – Chrome, Firefox, Opera, Edge, safari, etc.
    - ***Version control Compatibility testing:***
      - Validating application it support for different version on same browser.
      - Ex. Chrome browser – V90.10, V89.00, V85.00, V70.00, etc.

Configuration coverage testing/ hardware coverage testing:

- Validating the application I supporting the hardware (Print, Bluetooth, etc.).
- Configuration coverage testing also called hardware coverage testing.

Explain Globalization Coverage Testing in detail:

Definition: Globalization Testing ensures that the application supports multiple languages, regions, and cultural formats (like date, time, currency, and address formats) so that it can be used globally without functionality issues.
It checks whether the software can adapt to different languages and regions without any code changes.
Types of Globalization Coverage:

a. **Local (Localization Testing) :**

- Verifies that the application works correctly in a local or regional language.
- Focuses on content translation, cultural appropriateness, and local conventions.
- Example languages: Marathi, Gujarati, Punjabi, Bengali, etc.
- Example: Checking that all labels, buttons, and messages appear correctly in Marathi for users in Maharashtra.

b. **International (Internationalization Testing):**

- Ensures that the application can be easily adapted for any international language or country, like French, German, or Japanese.
- Focus is on verifying that the application design supports multiple character sets, date formats, and currencies.
- Example: Checking that the system supports € (Euro) for France and ¥ (Yen) for Japan.

c. **Global :**

- Verifies that the application supports a global (universal) language, typically English, which is understood and accepted worldwide.
- Ensures the app can be used globally without regional or cultural dependency.
- **For regional language validation, tools like Google Translator can be used for initial verification of translated content (though professional translation review is recommended for production quality).**

Explain Cookies and sessions in Detail:

Cookies:

- Temporary files created by the browser while browsing web pages.
- Used to store user preferences, login information, and session data for smoother navigation.

Sessions:

- Time-bound interactions created by the server when a user accesses the application.
- Sessions expire automatically after a certain period of inactivity to ensure security.
- Sessions help maintain user state (e.g., logged-in status) while navigating multiple pages.

Explain Security Testing in detail:

- **Definition:**
- Security Testing is a type of Non-Functional Testing that ensures the application is protected against unauthorized access, vulnerabilities, and threats.
- It checks how secure the application is from internal and external attacks
- Security testing is usually performed by skilled or specialized testers (ethical hackers / security analysts), not by general QA testers.

- **Objectives of Security Testing :**
- To identify vulnerabilities and weaknesses in the application.
- To ensure **data confidentiality, integrity, and availability.**
- To verify that authorized users have the correct access, and unauthorized users are restricted.

- **Authentication Testing :**
- Authentication is the process of verifying the identity of users — ensuring they are who they claim to be.
- It is the first step in any security process.

- **Common Authentication Methods:**
  1. **Username and Password: Users** enter valid credentials; if correct, they are granted access. Example: Login via email and password.
  2. **One-Time PIN (OTP):** Temporary code sent to email or mobile, valid for a single session. Example: OTP during online banking login.
  3. **Authentication Apps:** Use third-party apps (like Google Authenticator) to generate time-based security codes.
  4. **Biometrics:** Use physical identifiers like fingerprints, facial recognition, or retina scans. Example: Unlocking a phone using a fingerprint sensor.

- **Authorization (Access Control) Testing:**
  - **Definition:**
  - Authorization is the process of granting or restricting access to specific resources or actions after successful authentication.
  - It determines what an authenticated user is allowed to do.
  - **Examples:**
  - Granting a user permission to download a specific file from a server.
  - Providing admin access to certain users for performing system configurations.
  - Restricting normal users to read-only access in an application.
  - **Authorization always follows Authentication.**

Explain Re-testing in Detail:

- **Definition:** Re-testing is the process in which a tester verifies that previously reported defects (bugs) have been fixed correctly by the developer**.**
- It involves executing the same test cases again using the same test data to confirm the defect no longer exists.

- Re-testing is performed after a defect is fixed.
- It focuses only on the specific failed test cases — not on the whole system.
- It uses the same environment, same test data, and same test steps that were used when the defect was first found.
- **Purpose of Retesting :**
  - To ensure that the defect reported in the previous build is now resolved.
  - The functionality is now working as expected after the bug fix.
- **Test Data in Re-Testing:**
  - **From Databases (SQL Server):**
    - In existing or old projects, testers can use live or historical data available in the database by executing **SQL queries**.
    - Example: SELECT * FROM Users WHERE status = 'Active';
  - **For New Projects:**
    - If there is no existing data, testers create test data in the SIT (System Integration Testing) environment manually.
  - **Dependent Test Data:**
    - In some cases, test data depends on external systems or applications.
    - Example: In PhonePe, for testing UPI creation, data like mobile number, bank account, debit card details come from the banking system — provided by the Business Analyst (BA).
  - **Example Scenario:**
    a. **Build 1.0 Released:**
       - Test team found defects: Defect ID 1.0.1 and Defect ID 1.0.2.
       - Bugs were reported to the developer.
    b. **Build 1.1 Released (with bug fixes):**
       - Tester re-tests defects 1.0.1 and 1.0.2 using the same test data and steps.
       - If fixed → Tester **closes** the defect.
       - If not fixed → Tester **reopens** and assigns back to the developer.

Explain Regression testing in Detail:

- **Definition:** Regression Testing is a type of software testing performed to ensure that new code changes (additions, deletions, or modifications) have not adversely affected the existing functionality of the application.
- In other words, it verifies that the software still works correctly after updates, enhancements, or bug fixes.
- **Purpose of Regression testing: The** defect has been fixed properly and there is no negative impact on the existing features or interconnected modules after the change**.**
- **When Regression Testing is Performed :**
  - When a defect is fixed and a modified build is shared by the developer.
  - When **new features** are added or existing ones are updated.
  - When the application moves from one environment to another, such as **from SIT → UAT → Production.**

- **Example:**
  - If the developer fixes a login issue and modifies the login module, regression testing ensures that: **The login issue is fixed (re-testing)**, and other connected features like sign-up, password reset, or profile access are not broken due to the fix.

- **Formula:**
  - **Regression Testing = Regret + Action**
  - If a new change causes something to break, we "regret" the change and take "action" to fix it.

- **Types of Regression Testing:**

  1. **Unit Regression Testing:**
     - Focuses only on the specific code change or unit modified by the developer.
     - Performed in isolation without considering other modules.
     - Example: If only the "email validation" logic is changed, testing only that part.

  2. **Regional Regression Testing:**
     - Tests the modified module along with impacted modules identified through impact analysis.
     - An impact analysis meeting is held between QA and Developers to identify related areas.
     - Example: If the payment module is modified, regression also includes testing booking, invoices, and receipts.

  3. **Full Regression Testing:**
     - **Performed when there are multiple changes across modules or major releases.**
     - Instead of identifying specific impacted areas, the **entire application** is tested end-to-end.
     - **Example: Before a production release or after a major code overhaul.**

- **When Regression Testing is conducted:**
  1. **SIT (System Integration Testing) Environment:** When defects are found and fixed in SIT builds.
  2. **Final Regression**: Before moving the application from one environment to another (e.g., UAT → Production).

- **Regression Suite:**
  - **A Regression Suite is a collection of test cases that are re-executed during regression testing.**
  - **Test Cases Included in Regression Suite:**
    - Previously failed test cases (to confirm defect fixes)
    - High priority / critical test cases (core features).
    - Extra functionality test cases (new or changed features).

Explain the difference between regression and retesting:

Regression testing:

1. **Definition:** Regression Testing is a type of software testing performed to ensure that new code changes (additions, deletions, or modifications) have not adversely affected the existing functionality of the application.
2. To ensure that existing, unaffected features still work correctly after code changes.
3. **Purpose:** Ensures that new changes haven't broken old functionality.
4. Focuses on impact analysis and system stability.
5. **When It Is Performed:** Performed after any code change, new feature addition, or defect fix to confirm system stability.
6. **Test Case Scope:** Passed test cases from previous cycles, plus affected test cases, are re-executed.
7. Scope is wide and comprehensive.
8. **Test Data:** Can use old, modified, or new data depending on the functionality under test.
9. **Dependency:** Independent of defect fixes — performed even if no bug was reported, as long as code is modified.
10. **Execution Frequency:** Done multiple times — after every build or release cycle**.**
11. **Automation Possibility:** Highly suitable for automation, since regression suites can be reused across builds.
12. **Responsibility:** Can be performed by manual or automation testers to check system stability.
13. **Example Scenario:** After the login button fix, the tester also tests Signup, Forgot Password, and Profile modules to ensure no side effects occurred due to the fix.

Re-testing:

1. **Definition:** Re-testing is the process of testing a specific defect that was fixed to confirm it no longer exists.
2. To verify that the defect reported in the previous build is fixed.
3. **Purpose:** Ensures that defects are fixed properly by developers.
4. Focuses on defect verification.
5. **When It Is Performed:** Performed after a defect is fixed and a new build is received from developers.
6. **Test Case Scope:** Only failed test cases related to the fixed defect are re-executed.
7. Scope is narrow and focused.
8. **Test Data:** Uses the same test data used when the defect was originally found.
9. **Dependency:** Dependent on defect fixes — it's performed only after a bug is resolved.
10. **Execution Frequency:** Done once per defect fix**.**
11. **Automation Possibility:** Difficult to automate, as it depends on specific bug fixes and test data
12. **Responsibility:** Performed by manual testers to verify bug fixes.
13. **Example Scenario:** In Build 1.0, the "Login" button didn't work. After the developer fixes it in Build 1.1, the tester re-tests the same login test case.

Explain User Acceptance Testing (UAT) / Pre-Production Testing in detail:

- **Definition:** User Acceptance Testing (UAT) is the final phase of software testing conducted after System Integration Testing (SIT) and Business Black Box Testing (BBT).
- Its main purpose is to validate the end-to-end functionality of the application against the client's business requirements in a real-world environment before it goes live.
- UAT is also known as **Customer Acceptance Testing (CAT) or Pre-Production Testing.**

- **When UAT Starts:**
  - UAT begins after the successful completion of BBT in the SIT environment.
  - The SIT testers prepare test proofs (evidence of successful test execution) and send them to the UAT team.

- **UAT Team Composition:**
  - Typically, the UAT team consists of 2 Developers + 1 Tester (usually located at the client site).
  - The tester in SIT and the tester in UAT are not the same person — testers do not work in both environments.

- **Purpose of UAT:**
  - To verify that the system meets business requirements and is ready for production release.
  - To test the end-to-end flow of the application from the client's perspective.
  - To ensure that all functionalities, integrations, and data flows work as expected in a production-like setup.

- **Environment:**
  - UAT testing is performed in the UAT Server/Environment, which mirrors the production environment.
  - Testers generally access the UAT server through Remote Desktop Connection (RDP).

- **Types of User Acceptance Testing :**

  a. **Alpha Testing :**
     - Performed in a controlled environment also known **as in-premises testing.**
     - Conducted before the product is released to external users**.**
     - Done in collaboration with the client and the development team to validate the complete functional flow.
     - If any defects or change requests are identified during alpha testing, they are fixed immediately by the development team, since UAT happens before production deployment.
     - Example: Testing done at the software company's site with the client present to validate business scenarios.

  b. **Beta Testing :**
     - Performed in an uncontrolled environment also known as out-premises testing.
     - Conducted by actual end-users in a real-world environment.
     - The testing is done in a random manner, and user feedback is collected to identify issues or suggestions.

- Based on feedback, improvements or fixes are made to enhance user experience.
- Example: End users test a mobile app in their own devices and report bugs or usability issues.

- **UAT Defect Handling Process :**
  - If the UAT tester finds a defect, they report it to both the developer and SIT tester.
  - The SIT tester reproduces the issue in the SIT environment.
  - The SIT tester reproduces the issue in the SIT environment. The SIT tester informs the developer to fix it as soon as possible (ASAP).
  - If the defect is not reproducible in SIT. The SIT tester provides test proof (screenshots, logs, or video evidence) to the UAT tester as confirmation

Explain Hotfix or how to handle hot fix in detail?

Production testing or Hotfix:

- Production Testing, also known as **Hotfix Testing**, is conducted after deployment of the build from UAT (User Acceptance Testing) to the Production environment.
- Its main purpose is to verify and resolve defects that appear in the live (production) system, ensuring minimal impact on end users.
- A Hotfix is a quick patch or code update applied directly to the production environment to immediately resolve a critical issue.
- Also known as **QFE (Quick Fix Engineering) Update**.

- **Purpose of Hotfix testing:**
  - To ensure the stability, reliability, and functionality of the application in the live environment.
  - To verify that post-deployment fixes (hotfixes) or change requests (CRs) work correctly.
  - To confirm that no new issues have been introduced because of the fix.
  - To validate that production data, configurations, and integrations (like payment gateways, APIs, third-party services) are working correctly.

- **When Production Testing / Hotfix Happens :**
  - After deployment of the final build from UAT → Production.
  - When a production defect is reported by the client, end-user, or monitoring team.
  - When a critical business feature fails in production and needs immediate resolution.
  - When urgent enhancements or configuration changes are required.

- **Reasons for Production Defects:**
  a. **Missed Functionality by QA –** Tester did not test a particular scenario → Hotfix.
  b. **Missed Requirement by Client––** Requirement not captured earlier → Change Request (CR).
  c. **Environment Differences** – Production may have different data, configuration, or integration setup.
  d. **Deployment/Build Issues** – Incorrect version, missing configuration files, or

incomplete deployment.

e. **Data-Related Issues** – Live data may differ from test data used in SIT/UAT.
f. **Unstable Third-Party Dependencies** – Payment gateways, APIs, or external systems behaving differently in production.
g. **Concurrency or Load Issues** – Certain issues appear only under real-world load conditions.

- **Hotfix Process Workflow :**

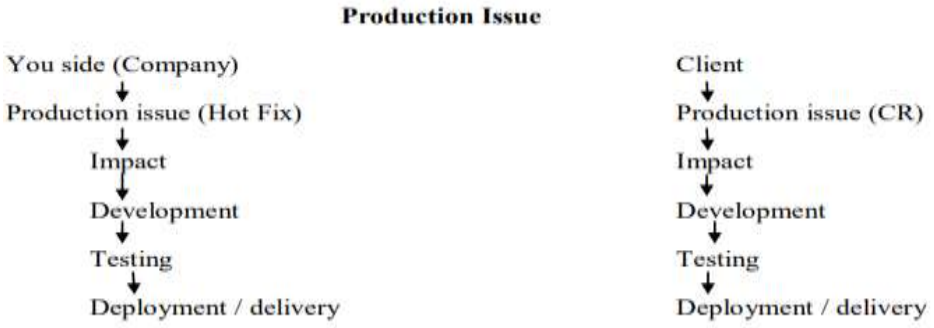
a. **Issue Identification:**
   - Client/end user reports a defect found in production.
   - Project Manager receives an escalation email from the client.

b. Issue Reproduction :
   - Tester attempts to reproduce the defect in the SIT environment using same steps, data, and configuration.
   - **Depending on the result:**
   - **Scenario A: Defect Reproducible in SIT**
     1. Tester reproduces the issue successfully in SIT.
     2. Reports issue to Developer with evidence (screenshots/logs/test proof).
     3. Developer fixes and shares the updated build.
     4. Tester performs Re-Testing in SIT → then Regression Testing to confirm no side impact.
     5. Once verified, build is deployed to production as Hotfix.
     6. Project Manager submits RCA (Root Cause Analysis) and communicates resolution to the client.
   - **Scenario B: Defect Not Reproducible in SIT**
     1. Tester is unable to reproduce the issue in SIT.
     2. Tester sends email to Project Manager, Developer, and Designer with test evidence
     3. Developer investigates:
        i. Deployment mismatch
        ii. Incorrect configuration/environment
        iii. Database or network issue
        iv. Server-side caching problem
     4. After investigation, Developer applies the fix directly in production if confirmed safe.
     5. QA validates the fix post-deployment (Production Smoke Test).

- **Change Request (CR) Handling Process :**

If client requests new functionality or modification:

1. Impact Analysis is conducted by Dev + QA + BA.
2. If high impact, CR is moved to next sprint or release.
3. If low impact, fix is implemented immediately → tested in SIT → deployed to production.
4. QA validates functionality through Production Testing.



Testing Terminologies:

- **Exploratory testing:**

- **Definition**: Exploratory Testing is an unscripted, experience-based testing approach where the tester simultaneously learns, explores, and tests the application to identify defects, understand the system behavior, and gain domain knowledge.
- **Purpose of exploratory testing :**
  a. To gain domain or product knowledge, especially for new team members.
  b. To explore unknown or less-documented areas of the application.
  c. To find critical or hidden defects that are often missed in scripted testing.
  d. To understand real-time application behavior, usability, and workflow.
- **When It Is Performed :**
  a. When a new tester joins the project and needs to understand the domain or product quickly.
  b. During initial phases of testing when documentation (like test cases or requirements) is incomplete.
  c. When there is limited time for formal test design and execution.
  d. To complement scripted testing — often used between test cycles or before regression.
  e. When requirements change frequently or are unclear.

- **Key Characteristics :**
  a) **Simultaneous Learning & Testing –** Tester explores the product while testing.
  b) **No Formal Test Scripts** – Relies on tester intuition and prior experience.
  c) **High Adaptability** – Approach changes based on findings and observations.
  d) **Focused Investigation** – Targets problem areas, complex features, or new functionalities.
  e) **Rapid Feedback** – Provides immediate insights to developers and product owners.

- **How It's Done :**
  a) **Start with a Charter –** Define the goal or area to explore (e.g., "Explore the booking flow for edge cases").

b) **Refer to Old Test Cases or Similar Modules** – Understand application logic or user behavior.
c) **Perform Ad-hoc Testing** – Try multiple combinations of inputs, edge cases, and workflows.
d) **Observe & Document Behavior** – Note any unexpected behavior, UI issues, or logical bugs.
e) **Share Findings** – Report defects or usability issues immediately to the QA lead or developer.

- ## Adhoc testing:

- Ad-hoc Testing is an informal and unstructured testing technique performed without predefined test cases or documentation.
- The tester relies completely on **domain knowledge**, product understanding, and experience to identify defects by exploring the application in a free-form manner.
- Testing is based on the domain or product knowledge only.
- In Adhoc testing, No any documentation, test case design etc.
- **Purpose of Adhoc testing:**
  a) To quickly identify defects in areas that might not have been covered by formal test cases.
  b) To verify application stability when formal documentation is missing or incomplete.
  c) To leverage tester's domain expertise for uncovering hidden or edge-case issues.
- **When to Perform :**
  a) When test cases are not yet prepared or documentation is missing.
  b) When time is limited, and testing must be done quickly (e.g., before a demo or urgent release).
  c) After major builds or hotfixes, to ensure basic functionality works as expected.
  d) To complement formal testing and enhance overall test coverage.
- **Key Characteristics :**
  a) **Unscripted & Unplanned –** No predefined test cases or test plan.
  b) **Domain Knowledge Required** – Tester must have good understanding of application and workflows.
  c) **Experience-Driven** – Tester uses experience to guess where issues might occur.
  d) **No Formal Documentation** – Defects are logged, but test steps are not predefined.
  e) **Highly Flexible** – Tester can focus on any area, feature, or use case spontaneously.

- ## Monkey Testing:

- **Definition:** Monkey Testing is an informal, random testing technique where testers input random data or perform random actions on the application to check its stability and find unexpected defects or crashes.
- The goal is to break the system and identify issues that occur under unpredictable usage scenarios.
- **Purpose of Monkey Testing :**
  a) To test the robustness and stability of an application.

b) To find hidden or unpredictable defects through random testing.
c) To stress the system by performing random and unplanned operations.
d) Commonly used when there's limited time for formal testing or test case execution.

- **When to Perform :**
  a) When there is very limited testing time before deployment.
  b) When test cases are not ready or requirements are incomplete.
  c) When builds are promoted quickly between environments (e.g., SIT → UAT or UAT → Production).
  d) Before urgent releases or client deliveries, to quickly validate system stability.
- **Key Characteristics :**
  a) Random Testing: No specific test cases or predefined inputs.
  b) Unpredictable Behavior: Testers perform unexpected actions intentionally.
  c) **Goal is to break the System:** Focus is on causing errors or crashes.
  d) **Domain Knowledge Required:** Tester must understand what random inputs or operations could stress the system most.
  e) **Unscripted Execution:** Tester uses creative thinking instead of structured plans.

Difference between Ad-hoc, Exploratory, and Monkey Testing:

Ad-hoc Testing:

a) Approach: Random but based on experience.
b) Domain Knowledge: Required
c) Objective: Find defects quickly.
d) Test Cases : None
e) Reproducibility : Moderate
f) **When Used:** No test cases / limited time.

Exploratory Testing:

a) Approach: Learning + testing simultaneously
b) Domain Knowledge: Optional
c) Objective: Learn and explore product.
d) Test Cases: Created dynamically.
e) Reproducibility : High
f) **When Used:** When requirements unclear.

Monkey Testing:

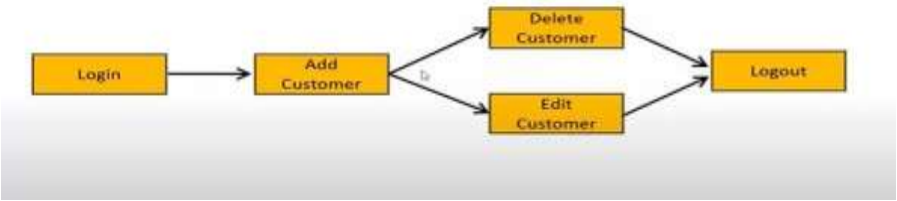
a) Approach: Fully random testing
b) Domain Knowledge: Required
c) Objective: Break the system.
d) Test Cases: None.
e) Reproducibility : Low
f) When Used: Before urgent release or to check robustness

End to End Testing :(Integration testing)

- **End-to-End (E2E) Testing** is a testing methodology used to validate the complete flow of an application, ensuring that all integrated components, modules, and systems work together as expected.
- It verifies the entire system functionality from start to finish, simulating real-world user scenarios.
- End to end testing is also cal**led as Integration testing.**
- Also known as **System Flow Testing or Business Flow Testing** in enterprise applications
- **Purpose of End to End Testing :**
    a) To test the overall business flow across multiple modules or systems.
    b) To verify that data flows correctly between integrated modules.
    c) To ensure end-user workflows perform as expected without interruption.
    d) To confirm that interfaces, APIs, databases, and dependencies function seamlessly.
- Integration testing will performed on complete module.
- Integration testing performed by Tester
- **Key Characteristics :**
    a) Covers complete business scenarios, not just individual features.
    b) Validates integration between modules (e.g., Login → Customer → Payment).
    c) Conducted after system testing is complete.
    d) Ensures data integrity and communication between components.
    e) Typically performed by **QA testers** (sometimes supported by automation).
- **Scope of End-to-End Testing:**

  It involves testing:

    a) Front-end to Back-end flow.
    b) Database and API communication
    c) External system integration (e.g., payment gateway, email service, third-party APIs)
    d) UI + Business logic + Data layer
- **End to End Testing is as follows:**
    - Login → User logs into the system using valid credentials.
    - Add new customer → User adds a new customer to the database.
    - Edit customer → User edits customer information.
    - Delete customer → User deletes the customer record.
    - Logout → User logs out successfully.



Explain Globalization Coverage Testing in detail:

Definition: Globalization Testing ensures that the application supports multiple languages, regions, and cultural formats (like date, time, currency, and address formats) so that it can be used globally without functionality issues.
It checks whether the software can adapt to different languages and regions without any code changes.
Types of Globalization Coverage:

d. Local (Localization Testing) :

- Verifies that the application works correctly in a local or regional language.
- Focuses on content translation, cultural appropriateness, and local conventions.
- Example languages: Marathi, Gujarati, Punjabi, Bengali, etc.
- Example: Checking that all labels, buttons, and messages appear correctly in Marathi for users in Maharashtra.

e. International (Internationalization Testing):

- Ensures that the application can be easily adapted for any international language or country, like French, German, or Japanese.
- Focus is on verifying that the application design supports multiple character sets, date formats, and currencies.
- Example: Checking that the system supports € (Euro) for France and ¥ (Yen) for Japan.

f. Global :

- Verifies that the application supports a global (universal) language, typically English, which is understood and accepted worldwide.
- Ensures the app can be used globally without regional or cultural dependency.
- **For regional language validation, tools like Google Translator can be used for initial verification of translated content (though professional translation review is recommended for production quality)**

Localization Testing:

- Performed to check system or software application for a specific geographical and cultural environment.
- Localized product only supports the specific kind of language and is usable only in specific region.
- Its testes the specific currency format, mobile number format and address format is working properly or not.
- For example, Baidu.com supports only the Chinese language and can be accessed only by people of few countries. Hence it is a localized product.

Explain Positive and Negative in Detail:

Positive Testing:

- **Definition**: Positive Testing is a type of software testing performed to verify that an application behaves as expected with valid inputs.
- The main goal of positive testing is to confirm that the system meets the specified requirements and produces the correct output when users perform valid operations.

- **Purpose of Positive Testing :**
- To ensure that the system works correctly for **valid and expected inputs.**
- To confirm that **normal user behavior** leads to successful execution without errors.
- To validate that the application's **functional requirements** are properly implemented.

- **Key Characteristics of Positive Testing :**
- Focuses on testing **intended functionalities** using **valid test data**.
- Helps confirm that the system performs as per **business rules**.
- Usually the **first phase** of functional testing.
- Does **not check for error handling** — only ensures that the system behaves correctly with valid inputs.

- **When It's Performed :**
  Positive testing is generally performed:
  a) During **Functional Testing and System Testing** phases.
  b) Before performing **Negative Testing**.
  c) In all testing levels (Unit, Integration, System, and UAT).

- **Example Scenarios :**

- **Example 1: Numeric Input Field**
  - There is a textbox in an application that accepts only numeric input up to **99999**.
  - **Positive Testing:** Enter values **from 0 to 99999** and verify the system accepts and processes them correctly.

- **Example 2: Online Payment Form**
  - User enters a **valid card number, expiry date, and CVV.**
  - **Positive Testing**: System successfully processes the transaction and shows confirmation.

Negative Testing:

- **Definition**: **Negative Testing** is a type of software testing performed to ensure that the application **handles invalid inputs and unexpected user behavior properly**.
- The main goal of negative testing is to **verify that the system does not crash** and **shows appropriate error messages** when invalid, incorrect, or unexpected inputs are provided.

- **Purpose of Negative Testing :**
- To check how the system behaves under **invalid input conditions**.

- To confirm that the application can **gracefully handle errors** and prevent system failures.
- To ensure that **validation rules**, **error handling**, and **data constraints** are implemented correctly.

- **Key Characteristics of Negative Testing :**
- Focuses on **negative or invalid scenarios**.
- Ensures that the system is **robust** and **stable** under invalid conditions.
- Performed after **Positive Testing** to validate negative behavior.
- Helps detect **data validation**, **security**, and **boundary-related defects**.

- **When It's Performed :**
  Negative testing is generally performed:
  a) After **positive testing** is completed successfully.
  b) During **System Testing**, **Integration Testing**, and **User Acceptance Testing**.

- Whenever an application involves **data validation or input handling**

- **Example Scenarios :**

- **Example 1: Numeric Input Field**
  - There is a textbox in an application that accepts only numeric input up to **99999**.
  - **Negative Testing:** Enter **alphabets (A–Z, a–z) or special characters (@, #, $)** and verify that the application rejects them or displays an error message such as *"Invalid input – only numbers allowed.*

- **Example 2: Email Field Validation**
  - System accepts **valid email formats** only**.**
  - **Negative Testing:** Enter invalid email like **user@com or user@@gmail.com** → System should display *"Please enter a valid email address.*

Basic Terms:

- **Error:**
  - **Definition:** An **Error** is a **mistake or fault** made by a **developer during coding** or programming.
  - It usually occurs due to **human mistakes**, **wrong logic**, **syntax issues**, or **incorrect calculations** during software development.
  - It can be done by developer during coding.
  - **Example :**
  - A developer writes a = b + c instead of a = b - c.
  - This causes incorrect functionality — hence, it's an **error** in code

- **Defects:**
  - **Definition**: A **Defect** is a **variation between the actual result and the expected result** observed during testing.

- It represents a **deviation from the client's requirement** or **functional specifications.**
- Example:
- Requirement: "Submit button should save the form."
- Actual: When clicking **Submit**, the form does **not save**.
  This difference is a **defect**.

- **Bug:**
  - **Definition:** When a **developer accepts a reported defect** as valid, it is then termed a **Bug**.
  - A **Bug** is an **accepted defect** in the software that needs to be fixed**.**
  - Also, if a **defect is missed by the tester** during testing and later found by the **end-user**, it is called a **Bug.**
  - **Example** :
  - Tester missed a login validation issue during testing.
  - End user reports that the system allows blank passwords.
  - That's a **bug** discovered after release.

- **Issue:**
  - **Definition:** An **Issue** refers to a **major bug or defect** that significantly impacts the functionality or stability of the application.
  - It may cause the system to crash, produce wrong outputs, or block testing completely.
  - Example:
  - Application crashes when user clicks "Save."
  - Payment gateway not working in production.
  - These are **critical issues**.

Difference between Functional and Non-Functional Testing:

Functional Testing	Non-Functional Testing
It verifies the **operations, features, and actions** of an application.	It verifies the **behavior, performance, and quality attributes** of an application
It is based on **business requirements** and validates *what the system should do*	It is based on performance and non-functional requirements, ensuring how the system performs
Examples: Unit Testing, Integration Testing, Regression Testing, Sanity Testing, Smoke Testing, User Acceptance Testing (UAT).	**Examples:** Performance Testing, Load Testing, Stress Testing, Security Testing, Usability Testing, Compatibility Testing**.**

Difference between smoke and Sanity Testing:

Parameters	Smoke Testing	Sanity Testing
Approach	It is a **broad and shallow testing approach** where all major modules and functionalities of the application are checked at a high level.	It is a **narrow and deep testing approach** where only specific functionalities of the application are tested
Purpose	Used to validate the stability of the build. If the smoke test fails, the build is rejected immediately and sent back to the development team.	The goal is to validate rationality and correctness of specific functionalities after minor changes, bug fixes, or small enhancements
Scope	Focuses on **critical and major functionalities** like login, navigation, and core workflow.	Limited to only those areas or modules where code has been modified.
Subset	Considered as a **subset of acceptance testing** because it ensures the build is stable enough to proceed with further testing.	Considered as a **subset of regression testing**, since it ensures that new fixes/changes do not break the related functionality.
Execution	Can be done **manually or automated**, depending on the project requirements.	Usually done **manually without detailed test cases or scripts**. Testers rely on their understanding of requirements and recent changes.
Performed By	Both **developers and testers**. Developers may run smoke tests before releasing the build, and testers validate again in QA.	Only by **testers** in the QA environment. Developers are not usually involved
Depth	Testing is done without getting into deep details, but if needed, testers may drill down to confirm critical failures.	Unlike smoke testing, sanity testing goes deeper into specific functionality, but does not cover the entire system.
When	Performed on every new build release.	Performed after receiving a stable build but with small modifications or fixes to confirm they work as expected.

Explain the difference between regression and retesting:

Parameters	Regression testing	Re-testing
Definition	Regression Testing is a type of software testing performed to ensure that new code changes (additions, deletions, or modifications) have not adversely affected the existing functionality of the application	Re-testing is the process of testing a specific defect that was fixed to confirm it no longer exists.
Purpose	Ensures that new changes haven't broken old functionality.	Ensures that defects are fixed properly by developers.
Focus	Focuses on impact analysis and system stability.	Focuses on defect verification
When It Is Performed	Performed after any code change, new feature addition, or defect fix to confirm system stability.	Performed after a defect is fixed and a new build is received from developers.
Test Case Scope	Passed test cases from previous cycles, plus affected test cases, are re-executed.	Only failed test cases related to the fixed defect are re-executed. Scope is narrow and focused
Test Data	Can use old, modified, or new data depending on the functionality under test.	Uses the same test data used when the defect was originally found.
Dependency	Independent of defect fixes performed even if no bug was reported, as long as code is modified	Dependent on defect fixes — it's performed only after a bug is resolved.
Execution Frequency	Done multiple times — after every build or release cycle.	Done once per defect fix
Automation Possibility	Highly suitable for automation, since regression suites can be reused across builds.	Difficult to automate, as it depends on specific bug fixes and test data
Responsibility	Can be performed by manual or automation testers to check system stability.	Performed by manual testers to verify bug fixes
Example Scenario	After the login button fix, the tester also tests Signup, Forgot Password, and Profile modules to ensure no side effects occurred due to the fix.	In Build 1.0, the "Login" button didn't work. After the developer fixes it in Build 1.1, the tester re-tests the same login test case.

Difference between Ad-hoc, Exploratory, and Monkey Testing:

Parameters	Ad-hoc Testing	Exploratory Testing	Monkey Testing
Approach	Random but based on experience.	Learning + testing simultaneously.	Fully random testing.
Domain Knowledge	Required	Optional	Required
Objective	Find defects quickly.	Learn and explore product.	Break the system
Test Cases	None	Created dynamically.	None
Reproducibility	Moderate	High	Low
When Used	No test cases / limited time	When requirements unclear.	Before urgent release or to check robustness.

Difference between alpha testing and beta testing:

Parameters	Alpha Testing	Beta Testing
Definition	Testing performed by internal team before release to end users.	Testing performed by real users before final product release.
Environment	Controlled (In-house / Company site)	Uncontrolled (User's environment).
Performed By	Internal testers, developers, sometimes client.	Real end users or customers.
Objective	Identify defects before releasing to users.	Evaluate product in real-world conditions.
Stage	Conducted after system testing, before beta testing.	Conducted after alpha testing, before final release.
Access	Limited access (pre-release build)	Public or limited external release.
Feedback	Collected from QA/test team.	Collected from real users.
Type of Issues Found	Functional and usability bugs.	Usability, performance, and real-world issues.
Duration	Shorter – within the company.	Longer – depends on user feedback.
Example	Internal QA tests a new payment flow.	Beta users test the payment flow before launch.

Priority and Severity in Software Testing:

- Both Priority and Severity are terms used to classify and manage defects effectively.
- They help teams decide **which defects should be fixed first** based on business and technical impact.
- Priority term defines defect **impact on client business.**
- Severity term defines defect **impact on functionality application.**

- ## Severity:

  - **Definition:** Severity defines **how serious a defect is from a functional or technical point of view.**
  - It is usually **set by the tester** based on how much the defect affects the system's functionality.
  - **Levels of Severity :**

    i) Critical :
    - Complete system failure or major module not working**.**
    - No workaround available**.**
    - Example: Application crashes, login not working.

    ii) High :
    - Major business functionality is impacted.
    - Users cannot perform important action.
    - **Examples:**
    - Fund transfer not working in banking app.
    - Product checkout not working in e-commerce.

    iii) Medium :
    - Some undesirable behavior occurs, but the application is still functional.
    - **Examples:**
    - No confirmation message after sending email.
    - No booking confirmation after scheduling a ride.

    iv) Low:
    - Cosmetic or UI-related issue; does not affect functionality.
    - **Examples:**
    - Spelling mistake, alignment issue, or logo misplacement.

- ## Priority:

  - **Definition:** Priority indicates **how quickly a defect should be fixed** based on business or customer needs.
  - It is usually set by **the project manager or product owner** based on business impact.
  - **Levels of Priority :**

    i) High :
    - Defect must be fixed immediately as it blocks business operations**.**
    - Example: Login page not loading, payment failure.

    ii) Medium :
    - Fix can wait until the next release or build.
    - Example: Minor form validation issue**.**

iii) Low :

- Fix can be deferred to later releases; not urgent.
- **Examples:**
- Typographical or UI color inconsistency.

- **Examples of Severity and Priority :**
  - **Low priority-Low severity:**

- A spelling mistake in a page not frequently navigated by users.
- Ex. Spelling mistakes in application (Ex. Submit text changes – Ok text)
- Ex. Button color is not proper (Ex. Button Blue color into red color)
- Any cosmetic or spelling issues which is within a paragraph / page.

  - **High Severity- Low Priority:**

- Application crashing in some very corner case.
- Application rarely used functionality is not working (Ex. Paytm –Promo Code).
- Invoice download is not working in application.
- Web page not found when user clicks on a link (user does not visit that page generally).

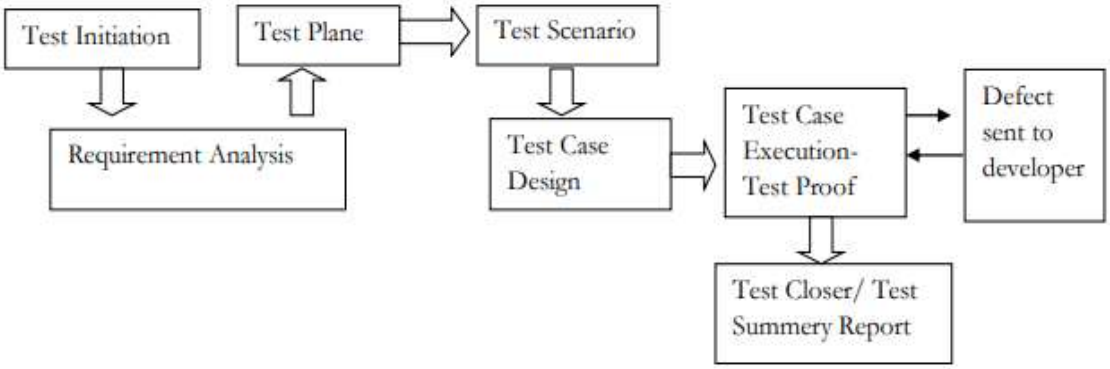
  - **High priority-Low severity:**

    - Slight change in logo color or spelling mistake in company name.
    - Application text are overlapped with each other.

  - **High priority-High severity:**

    - Login page is not working- defects**.**
    - Core functionality application in project – defects.

What is your organization Test process?
Software Testing Life Cycle (STLC)



- **Test Initiation:**
  - In test initiation stage Project Manager is involved.
  - Project Manager initiates the testing process after receiving project requirements.
  - Prepares a **Test Responsibility Matrix (TRM)** — defines roles, responsibilities, and ownership for all testing activities.
  - After completion, the TRM is reviewed and shared with the Test Lead for planning the next stages.
  - **Deliverables:** Test Responsibility Matrix (TRM), Initial Test Planning Inputs.

- **Requirement Analysis:**
  - In this stage project manager and test lead involved.
  - Analyze business requirements, user stories, and acceptance criteria from the requirement document (BRD/FRS).
  - Identify **testable and non-testable** requirements.
  - Decide **testing strategy** and **testing methodology** based on application scope.
  - **Strategy:** Defines the technology stack (e.g., Java, Python, Ruby, C#).
  - **Methodology:** Defines the testing approach (Manual, Automation, API, Database Testing, etc.).
  - Determine test environment needs and identify any risks or dependencies.
  - **Deliverables:** Requirement Traceability Matrix (RTM), Testing Strategy and Methodology Document.

- **Test Plan:**
  - Test plan will prepared by test lead.
  - Prepare a detailed **Test Plan Document** based on the analysis phase.
  - Define the **scope of testing**, **objectives**, **resources**, **schedule**, **test deliverables**, and **risk mitigation plan**.
  - Allocate **testers, tools, and timelines** for the testing activities.
  - Define entry and exit criteria for test execution.
  - Test plan will contains – who will test, when to test, how to test etc.
  - **Deliverables :** Approved Test Plan Document , Resource and Effort Estimation Sheet

- **Test Scenario & Test Case Design:**
  - In this Phase, tester and QA Team is involved.
  - Identify test scenarios from user stories or requirements.
  - Create detailed **test cases** covering all positive and negative test conditions.

- Prepare **test data** for each case.
- Test cases are reviewed by the Test Lead and documented in Excel or Test Management tools.
- **Deliverables :** Test Scenarios Document , Test Case Document (TCD), Test Data Sheet

- **Test Case Execution:**

  - In this Phase, tester and QA Team is involved.
  - Receive the build from the development team and verify build stability (Smoke/Sanity testing).
  - Execute test cases based on test plan and test data.
  - Log defects in tools like **JIRA** or **TFS**, including screenshots and reproduction steps.
  - Communicate defects to the development team via email or defect tracking comments.
  - After fixes, perform **Re-testing** to validate defect fixes and **Regression Testing** to ensure stability.
  - **Deliverables:** Executed Test Case Report, Defect Report, Re-test & Regression Logs.

- **Test Closure/ Test Summary Report:**

  - Test closure report will prepared by test lead
  - Summarize sprint-wise testing activities such as :
    (1) User Stories (US) covered
    (2) Test Cases Designed (TCD)
    (3) Test Cases Executed (TCE)
    (4) Test Cases Passed (TCS)
    (5) Defects logged, closed, and deferred
  - Obtain formal sign-off from the Project Manager or Client.
  - **Deliverables:** Test Summary Report (TSR), Test Metrics Document, Sign-off Approval

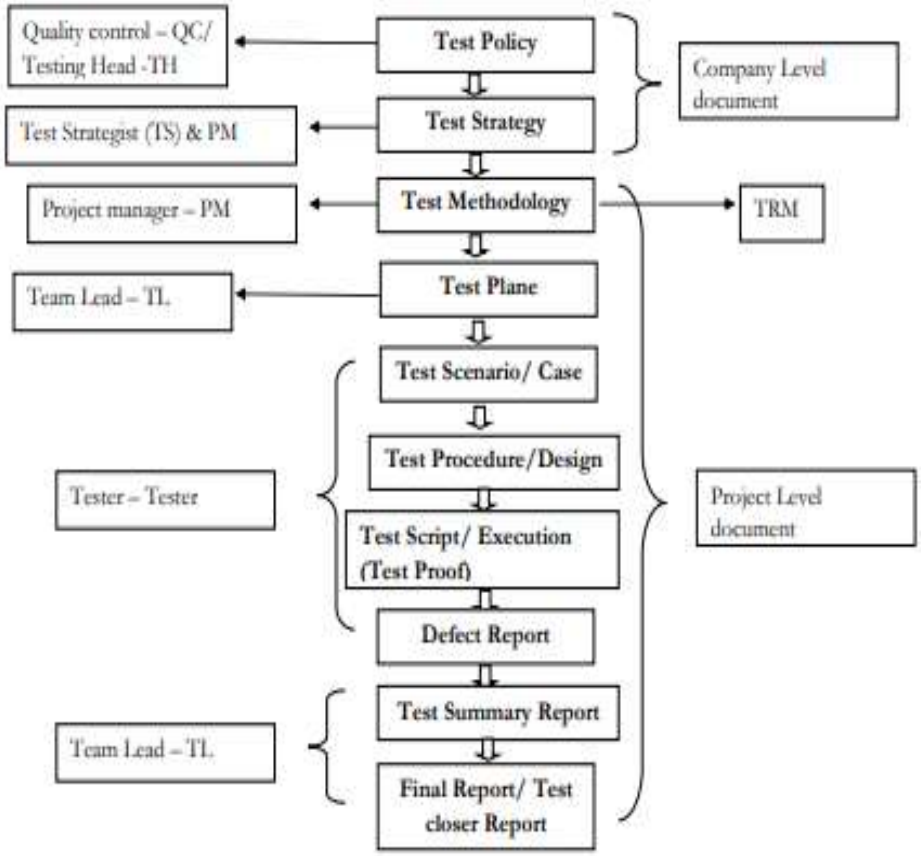
What is your organization Test documentation?

- **Definition:**

- Test documentation refers to the set of documents that describe the testing process, strategy, scope, test cases, and reports followed throughout the Software Testing Life Cycle (STLC).

- It ensures transparency, consistency, and traceability in QA activities across all projects

- **<u>Test document hierarchy</u>**



- **Test Policy:**

  - Test Policy document prepared by **Testing Head / Quality Control Team.**
  - Test Policy document is **company level documents.**
  - **Purpose of Test Policy Document:**
    a) Defines the **testing objectives**, **scope**, and **overall goals** of the organization's QA process.
    b) Establishes **quality standards** and **compliance metrics** to be followed across all projects.
    c) Outlines how QA contributes to overall business value (e.g., customer satisfaction, reduced production defects, cost reduction).
    d) Defines how much revenue or business impact is expected from successful QA delivery.
  - **Example:** The test policy ensures that every project undergoes at least one round of UAT before deployment to production**.**

- ## Test Strategy:

  - Test Strategy document prepared by **Test Strategist / QA Manager.**
  - Test Policy document is **company level documents.**
  - **Purpose of Test Strategy Document:**
    a) Defines the **overall testing approach** to be followed in the organization.
    b) Specifies **testing types** to be used (manual, automation, performance, security, etc.).
    c) Defines **test levels** (unit, integration, system, UAT).
    d) Describes **test environment setup**, **test tools**, **defect management**, and **reporting process.**
    e) Ensures consistency and uniformity in testing across multiple projects.
  - **Example:** All client-facing modules must undergo regression and security testing before release.

- ## Test Methodology

  - Test Methodology document prepared by **Project Manager.**
  - Test Methodology document is **Project Level Document.**
  - **Purpose of Test Methodology Document:**
    a) Defines the **mapping between development stages and testing activities**.
    b) Specifies **which testing phase aligns** with each SDLC stage**.**
    c) Focuses on the **testing model** (V-Model, Agile, Waterfall, and Hybrid).
    d) Identifies **project-specific risks**, dependencies, and mitigation plans.
    e) Includes preparation of the **Test Responsibility Matrix (TRM)** (also called Test Matrix).
    f) TRM defines who is responsible for testing each module, functionality, or requirement.
    g) While preparing TRM document, focus on parameters
    h) Functional / requirements of application
    i) Risk in the Project.

Development (Stage) → **Testing factor ↓**	BRS, SRS	Design	Coding	Testing	Support
Requirement phase Testing	Yes	Yes	Yes		
UI Testing	No	Yes	Yes	Yes	Yes
Functional Testing	No	No	Yes	Yes	Yes

- ## Test Plan:

  - Test Plan document prepared by **Test Lead.**
  - Test Plan document is **Project Level Document.**

- **Purpose of Test Plan Document:**
    a) Outlines the complete **plan of testing** for the project.
    b) Specifies **testing objectives, scope, schedule, resources, and tools.**
    c) Defines **test entry and exit criteria.**
    d) Contains **job allocation**, **test environment setup**, and **risk mitigation plans.**
    e) Test plan will contains – **Job allocation, resource allocation & estimations.**
    f) Test plan will contains – **who will test, when to test, how to test etc.**

Test Scenario & Test Case Design:

- Test Scenario and Test Case Design document prepared by **QA Testers.**
- Test Plan document is **Feature / Module Level Document.**
- **Purpose of Test Scenario & Test Case Design Document:**
    a) Identify **test scenarios** based on user stories or requirements.
    b) Design **test cases** covering all positive, negative, and boundary conditions.
    c) Maintain test cases in **Excel or Test Management tools** like TestRail, Zephyr, or QTest.
    d) Prepare test data and obtain lead approval before execution.
- **Deliverables :** Test Scenario Document , Test Case Document (TCD), Test Data Sheet

Test Case Execution:

- Test Case Execution document prepared by **QA Testers.**
- Test Case Execution document is **Project Level Document.**
- **Purpose of Test Case Execution Document:**
    a) Execute test cases after build deployment.
    b) Log defects in tools like **JIRA** or **TFS**, attaching screenshots and reproduction step.
    c) Communicate issues with developers and retest once fixes are deployed.
    d) Perform **Regression Testing** to ensure stability.

Defect Report:

- Defect Report document prepared by **QA Testers.**
- Defect Report document is **Project Level Document.**
- **Defect Report Includes:** Defect ID , Summary & Description, Severity & Priority, Status (New, Open, Fixed, Closed, Reopened), Module Name, Environment, Reported By / Assigned To

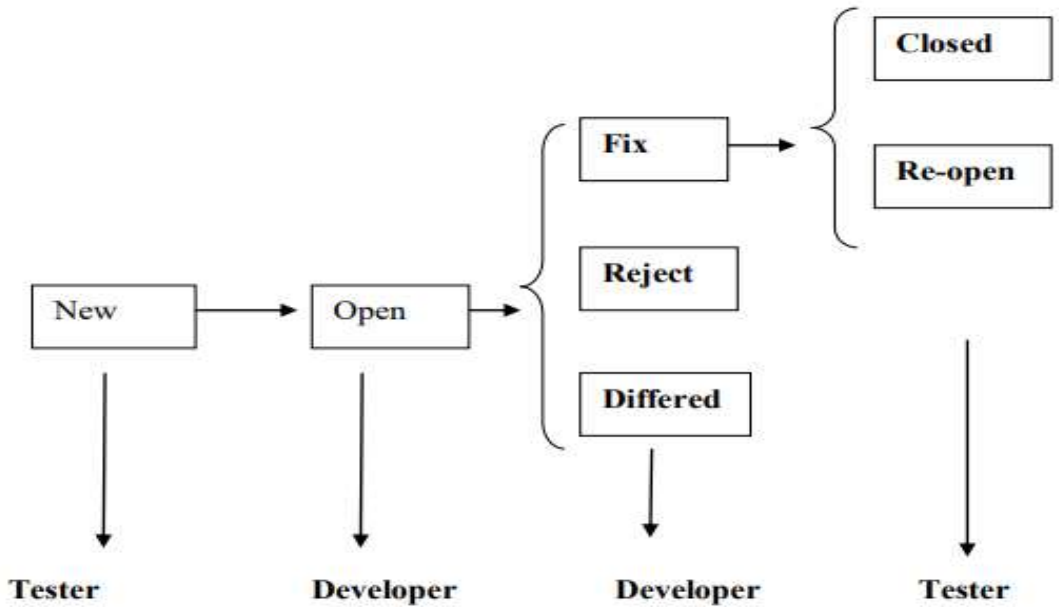
Test Closure/ Test Summary Report:

- Test Closure/ Test Summary Report document prepared by **QA Lead.**

- Test Closure/ Test Summary Report document is **Sprint or Release Level Document.**
- **Purpose of Test Closure/ Test Summary Report:**
    a) Summarizes all testing activities and results after completion of a testing phase or sprint.
    b) Includes: Test Cases Designed (TCD), Test Cases Executed (TCE), Test Cases Passed (TCS), Test Cases Failed (TCF), Total Defects Logged, Closed, Deferred, Sprint-wise test coverage.
- Includes final sign-off from QA and Project Manager.

Explain defect or bug life cycle in detail?

- **Defect:** When a **tester finds a deviation** in the application that does not match the **expected result** or **requirement**, it is called a **defect**.
- **Defect life cycle:** The journey of the defects from their start to end (New to close) it is called as defect life cycle.
- The journey of defect is goes like from **New** >>**Assign**>>**open**>>**fixed** >>**retest**>>**verified**>>**closed.**



- **New:**
  - When tester will found defect in the application or build then tester will log defect in a Jira and **mark defects status as New.**
  - We will assign that defect to the developer at that time **defect status as Assign.**
  - Tester will inform to developer through mail by JIRA.

- **Open:**
  - When developer start analyzing that defect that time developer will mark the **defects status as Open.**

- **Fix :**
  - When developer found it is valid defect then developer will fix and sent to the tester and **mark defects as Fix.**
  - Developer will inform to tester through email.

- **Retest:**
  - On the fixed defects, tester will do testing and regression testing to ensure that functionality is working fine or not. **At the time defect status as Retest.**

- **Verified:**
  - Once we are done with retesting and ensure functionality is working fine or defect is no longer present in the build **at that time defect status as Verified.**

- **Closed:**
  - When verification of the defect is done and once all the functional scenario is working fine at that time we will **change the defect status as closed.**

- **Re-open:**
  - On the fixed defects, tester will do retesting and regression testing.
  - If he found defect have not been fixed by developer.
  - Functionality is not working fine and **mark defect status as Re-open.**
  - Tester will inform to developer through email.

- **Duplicate:**
  - If the same defect is logged multiple time or it is corresponds to same functionality at that time, developer **mark the defect status as Duplicate and it is closed.**

- **Reject:**
  - When developer checking defect and he found defect is invalid defect and **mark defect status as Rejected.**
- **Deferred:**
  - When developer found, these defect can't be fixed in current sprint as it requires more time to fix.
  - At that time developer **mark the defect status as deferred.**
  - Defect status as deferred will be decided by project manager, business analyst and designer.
  - Project manager will prepare a user story for these deferred **defect as "Carried over bug"**
- **Re-producible defects:**
  - Re-producible defects are those defects which is producing again and again.
  - For re-producible defect, In JIRA tool we will **marked as Re-producible defects =Y/N.**
- **Bug Leakage:**
  - Defect those found which is missed from SIT environment and found in UAT environment.
  - Tester in SIT environment, he will reproduce that defects.

What exactly defect report contains?

- **Defect Report Contents:**
- **Defect_I0:** Unique identification number for the defect.
- **Detect Description:** Detailed description of the defect including information about the module inwhich defect was found.
- **Version**: Version of the application in which defect was found.
- **Steps:** Detailed steps long with screenshots with which the developer can reproduce that defects
- **Date Raised:** Date when the defects raised.
- **Reference**: where you Provide reference to the documents like. Requirements, design, architecture or may be even screenshots of the error to help understand the defect.
- **Detected By:** Name/ID of the tester who raised the defect.

- **Status**: Status of the defect, more on this

  later.

- **Fixed by:** Name/i0 of the developer who fixed it.

- **Date closed:** Dat**e** when the defect is closed.
- **Severity :**Which describes the impact of the defect on the application
- **Priority:** which is related to defect fixing urgency. Severity Priority could be High/Medium/Low based on the impact urgency at which the defect should be fixed respectively.

What are the 7 principles of software testing?

- **Present of defect in testing**
  - Test the software in order to find the defects.
  - Testing can uncover defects, but it cannot guarantee that the software is completely free of defects.
  - The goal of testing is to reduce the number of undiscovered issues, not to prove perfection.
- **Early Testing**
  - Start software testing at early stages means from the beginning when you get requirements.
  - Testing activities should start as early as possible (example requirements review, design review).
- **Defect clustering**
  - A small number of modules usually contain most of the defects.
  - Testing should prioritize high-risk and defect prone areas.
- **Exhaustive testing**
  - Should not do Exhaustive testing. Means we should not use same type of data for testing everytime.
  - It is not feasible to test all possible inputs, scenarios, and paths.
  - Instead, testing should focus on risk-based, priority-based and representative test cases.
- **Pesticide Paradox**
  - We should follow the concept of Pesticide Paradox means, if you are executing same cases for longer run, they won't be find any defects.
  - We have to keep update test cases in every cycle/release in order to find more defects.

- **Testing dependency**
  - Testing is context based means decide what types of testing should be conducted based on type of application.
  - There are several domains available in the market, such as banking, medical, advertisement, etc. Each domain has its applications with different requirements, functions, and testing purposes. In a nutshell, different domains, different testing.
  - For example, safety-critical software is tested differently from an e-commerce website. The risk associated with each type of application is different, so using the same method is ineffective
- **Bug/ Error free application (95% < defects Build reject) Absence-of-errors:**
  - We should follow **defect clustering**. Means some of the modules contains most of the defects.
  - Just finding and fixing defects does not guarantee that the system meets user needs or is useful.
  - The software must also fulfil business requirements and expectations
  - By experience, we can identify such risky modules. 80% of the problems are found in 20% of the modules.

Explain the traceability matrix?

- In the traceability matrix the **Test cases & defects are linked/mapped with user story.**
- A **Traceability Matrix (TM)** is a document used to ensure that all the requirements defined for a system are tested thoroughly through test cases.
- It establishes a **relationship (mapping)** between **requirements, test cases, and defects** to make sure **no functionality is missed during testing.**
- **Purpose of Traceability Matrix :**
  - To ensure **100% test coverage** — all requirements have corresponding test cases.
  - To **track the progress** of testing activities.
  - To verify that **each defect** is linked back to a **specific requirement or user story.**
  - To easily **identify missing or redundant test cases.**
  - To help in **impact analysis** when requirements change.
- **2 types Traceability matrix:**
  - **Forward Traceability matrix-.**
    - It ensures that **each requirement/user story** has corresponding **test cases** written for it.
    - In forward traceability matrix, **test cases should be link/mapped**

with the user story.
    - Test cases written in excel sheet, these excel sheet attached with user story.
- **Backward Traceability matrix:**
    - It ensures that **each test case and defect** can be traced back to a **requirement/user story.**
    - In Backward Traceability matrix, **defects should be link/mapped** with the user story.
    - Defects will be created in JIRA/ TFS tool.
    - In JIRA/ TFS tool defects link with user story.

What is review?
What are the different types of review followed in your organization?

- Review is the process of **verifying the correctness and completeness of documents**, such as test cases, requirements, or design documents.
- **Types of Reviews :**
    - **Self-review:**
        - Tester will do test cases review, these review is called self-review.
        - Ensures initial accuracy and completeness.
    - **Peer review:**
        - Test Lead or Senior Tester reviews the test cases.
        - Tester shares test cases via email with the reviewer.
    - **Internal review:**
        - Business Analyst (BA) reviews the test cases.
        - Tester sends the test cases to the BA.
        - BA schedules a meeting with the tester to review and discuss the test cases.
    - **External review:**
        - Client or UAT tester reviews the test cases.
        - Tester informs the Test Lead, who sets up a meeting with the client/UAT tester.
        - Tester presents and explains the test cases during the meeting.

Explain test plan (Test Plan) that followed in your organization?

- Test plan will be prepared by Test lead.
- **While preparing Test plan will consider following point.**

  - **Job allocation:**
    - Assigning specific task or responsibilities to team members
    - Team lead assign work to individual tester like who work in SIT, Regression and UAT environment.
    - Purpose in Test Plan: Ensures clarity on who will do what so there is no overlap or confusion.

  - **Resource allocation:**
    - Planning and assigning the resources (human resources, tool, environments, device, test data) required to execute the testing activites
    - Also, check environment availability to perform testing to achieve the object of product.

  - **Estimation:**

    - Predicting the time, effort, and cost required for test planning, test case design, test case execution, defect reporting, and regression cycles.

  - **Main aim of test plan :**
    - To decide start and end of the testing.
    - To provider a systematic approach for testing by defining:
      1. *Scope* (what will and won't be tested)
      2. *Objectives* (quality goals)
      3. *Resources & Schedule* (who, when, and with what)
      4. *Responsibilities* (job allocation)
      5. *Strategy* (how testing will be carried out)

  - **Test plan contains:**

    - **Test plane ID:**
      - Test lead create an Id for the modules they have tested.

    - **Test items:**
      - Test items defines the software components, modules, or features that are subject of testing.

- **Iteration:**
  - Iteration refers to a cycle or round of testing that repeats within a project's test execution timeline.
  - It is usually aligned with **agile sprints**, **release cycles**, or **phases** of development.
  - Each iteration may include:
    - Planning
    - Test design
    - Test execution
    - Defect reporting & fixing
    - Re-testing / regression

- **Feature to be tested:**
  - The list of all the features within the scope are to be tested will be listed out here in this.

- **Feature not to be tested:**
  - The list of all the features that are not for planned for the testing will be listed out here in this.

- **Test pass/ fail:**
  - Decide the criteria for test case execution.

- **Test deliverable**
  - The lists of all the documents that are to be prepared during testing process will be maintained here in this section.
  - Ex: Test Plan Document, Test Scenarios & Test Cases (Excel file). Test Data, Defect / Bug Reports (logged in Asana/Jira), Traceability Matrix (RTM), Test Execution Reports (Daily/Weekly), Test Summary Report (Closure Report) etc.

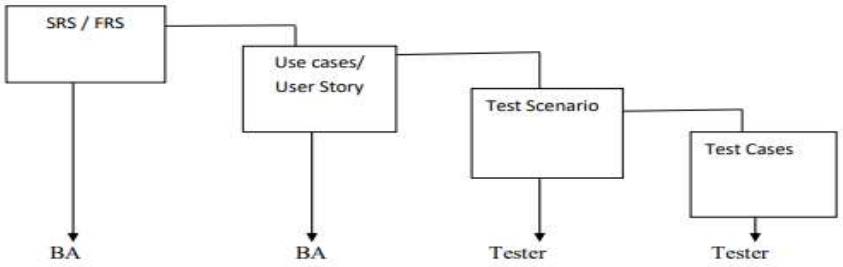
- **Test Environment:**
  - The clear details of the environment that is about to be used for testing the application will be maintained in this section.
  - Ex: Depending upon the Project the Environment will be selected.
  - Hardware (PCs , mobile devices , tablets)
  - Software(OS,Browser,Apps)
  - Test Servers/Databases
  - Network Configurations
  - Test Tools (Selenium, Postman, Jmeter etc.)
  - Environment Type (Dev , QA, UAT , Production)

- **Testing Task:**
  - Who has to do what will be clearly planned and maintained.

- **Staff training (New module):**
  - To accomplish this project successfully any staff or training is required then that information will be clearly maintained in this section.

- **Sign Off & Approval:**
  - Who has approved this document, when it is approved will be maintained in this section.
  - **Ex: Test Manager.**

Explain the terms test scenario and test case in detail?



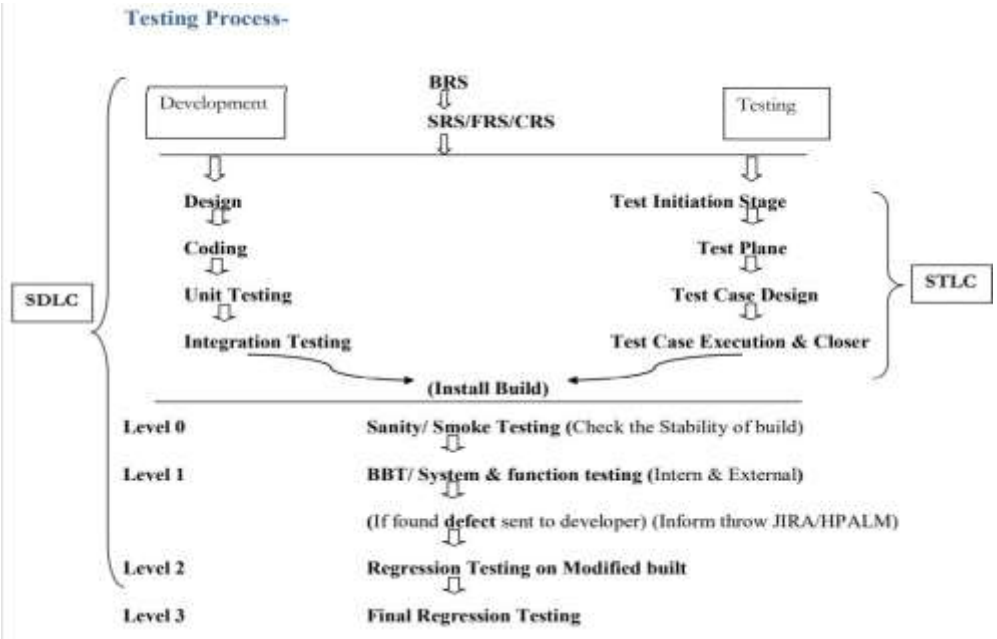
- **Software Requirements specification :(SRS)**
  - SRS defines functional requirement to develop & system requirement that will be used.
  - SRS will be prepared by business analyst.
  - SRS derived from business requirement specification document.
  - SRS will contains:
    1. Functional requirement
    2. Functional flow diagram
    3. Use case
    4. Screenshot.

- **Use Case:**
  - Use cases defines a single/ specific requirement.
  - Use cases will be prepared by business analyst
  - Use cases derived from SRS.
  - Use cases will contains
    - Description
    - Acceptance criteria.

- **Test Scenario:**
  - Test scenario will be prepared by tester against the user story.
  - Test scenario will be derived from US.
  - Test scenario defines **"What to Test".**
  - Test scenario defines ways to test the functionality.
  - Form one test scenario, multiples test cases will prepared.
  - Test scenario defines the high level test cases.
  - Test scenario always defines in positive ways.

- **Test case:**
  - Test cases will be prepared by tester against the test scenario.
  - Test cases will be derived from test scenario.
  - Test cases defines ***"How to Test".***
  - Test cases defines input, process, and output.
  - Form one test cases, test data, result, etc.
  - Test cases defines the low level test cases.
  - Test cases always defines in positive way & negative way.

Explain the development and testing process that followed in organization?



Questions on process model?

1) **How will you receive the project requirements?**
   - The finalized SRS will be placed in a project repository; we will access it from there.

2) **What will you do with SRS?**
   - SRS stands for software requirement specification.
   - SRS is used to understand the project functionality from business and functional point of view.

3) **Is the testing team involved in SRS preparation?**
   - Business analyst prepare the SRS document by interacting with the client.
   - However a senior testing team member can also be involved in requirements collections along with the development team and the business analyst team.

4) **How does your requirements document look like?**
   - It contains lots of use cases where each use case explains one or more functionalities.

5) **How will you understand the requirements?**
   - If it is known domain by going through use cases I can understand the requirements.
   - If I have some queries, I will discuss them with business analyst for clarifications.
   - If it is new domain, first I will get domain training them I go through the use cases.
   - If the project requirements are very confusing, then (BA) can also walk through each use case.

6) **How do u understand functionality without screens?**
   - We get wireframes in the use cases which helps a lot to understand the functionality.

7) **What is wireframe?**
   - A diagram which stimulates the feel of the actual screen.

8) **What is use case?**
   - Use case explains the step by step procedure of how a particular functionality of s/w is used by the end user.
   - **Use case contains sections such as.**

     Use case id.
     Use case name.
     Description.
     Flow of events.
     Alternative flow of events.
     Pre, post conditions

9) What will be the problem without SRS?

- Without SRS we will not be able to understand the project features correctly. Hence we will not able to test the project in depth and deliver the best quality product.

10) Where you involved in writing the use cases?

- I am aware of how use cases looks like and I can write if required.
- But I have never got an opportunity to write the use cases because these are prepared *by requirements gathering team.*
- Any how I have reviewed the use cases of certain functionalities and have given my inputs for betterment of the same.

11) How long do u spend on understanding SRS?

- It depends on the familiarity of the domain and complexity of the project.
- If it is a familiar domain, we can understand around 25 pages of the documentation every day.
- For a new complex domain, we manage around 15 pages per day**.**

12) What are the different sections present in SRS?

- Scope
- Features
- User characteristics
- Software requirements
- Hardware requirements
- Performance requirements
- Use cases
- Security and reliability requirements

Questions on Agile Methodology?

1. **What are the different artifacts in agile?**
   - Product backlog, sprint backlog, burndown chart, burn up chart etc.

2. **How you will decide the estimation in Sprint planning meeting?**
   - In my project, PM will open the Agile Board (JIRA/HAPLM) in sprint planning meeting.
   - PM will ask to every developer & tester about the time/ story point for every US which assigned to you.
   - Estimation will defines depends upon the number of factors:

     How much knowledge you have against user stories.

     How much efforts will be required.

     How much complexity of the user stories.

3. **What is an Agile?**
    - Agile is an iterative approach to software development methodology using short iteration of 1to 4 weeks.
    - Due to the agile methodology, the development process is aligned to deliver the changing business requirements

4. **Explain velocity in agile?**
    - Velocity is a matrix that is calculated by the addition of all effort estimates associated with user stories completed in an iteration.
    - It predicts how much work agile can complete in a sprint and how much will it require to complete a project.

5. **When do we use agile scrum methodology?**
    - When client in not clear on requirements
    - When client expects quick release
    - When client doesn't give all the requirement at a time

6. **What is a sprint retrospective meeting?**
    - Scrum team meets again after the sprint review meeting and documents the lessons learns in the earlier sprint such as "What went well" and what could be improved

7. **What is a sprint review meeting?**
    - In the Sprint review meeting, scrum development team present a demonstration of potentially shippable product.
    - Product owner declares which items are completed and not completed.
    - The product owner add the additional items to products backlog as per stakeholder's feedback.

8. **What is difference between agile and scrum?**
    - Agile is a practice and scrum is a model. So, these scrum people do the agile practice and that agile practice follows some principles of agile software development Approach. And those principles are.
        a. **Testing moves the project forward:** Continuous testing is the only way to ensure continuous progress.
        b. **Testing is not a phase:** Agile team tests alongside the development team to ensure that the features implemented during a given iteration are actually done.
        c. **Everyone tests:** In agile testing, the entire team including analysts, developers, and testers test the application. After every iteration, even the customer performs the User Acceptance Testing**.**
        d. **Keep the code clean:** The defect are fixed as they raised within the same iteration.
        e. **Lightweight / Less documentation**

f. **Test Driven Development Approach (TDD):** Test Cases are written along with the requirements. Hence, development can be driven by testing. This approach is called Test Driven Development (TDD) and Acceptance Test Driven Development (ATDD).

9. What is a sprint planning meeting?

- The first steps of scrum is the sprint planning meeting where the entire scrum team attends.
- Here the product owner selects the product backlog items (user stories) from the product backlog
- Most important user stories at the top of list and least important user stories at the bottom.
- Scrum development team decide and provide estimation.

10. What is Sprint zero?

- It can defined as pre-preparation step to first sprint. Activities like setting development environment, preparing backlog etc. need to be done before starting the first sprint and can be treated as sprit zero.

11. When are a delivery/ deployment in your project?

- In my project we are working in Scrum Agile methodology, where we will work for two week/ three week (Monday to Friday).
- In my project we will Delivery/ deployment on – Saturday/ Monday.

12. Who will do the Delivery/ deployment?

- In my all project, developer will do deployment to all environment & to the client.

13. What is your approach if US is not completed (development/ testing) in current Sprint?

- In my project, PM will tell to developer & tester will work on Saturday & Sunday andcomplete the work (More chances work will be complete).
- But still If still work will not completed in Saturday & Sunday then PM will prepared a US innext sprint – US= "Reaming Work In Last Sprint"

14. Different types of technology in your project

- Frond end is Dot net languages
- Services is Java languages
- Backend/ Database is SQL Server

15. **What is your approach, when SIT environment is crash?**

- If client want Deployment with the sprint and SIT environment is crash, then tester will access the UAT environment.
- UAT environment access throw remote desktop.

16. What is your approach, defect is not fixed in current sprint? Or if we got defect in last 1hr in last day?

- If we got issue/ defects (high priority/ core functionality) in last of sprint in testing.
- Developer & tester work extends (Friday) – try to fix & tester will test.
- If in Friday, work is not completed so developer & tester have to work on Saturday & Sunday.
- There will more chances to complete the user story.
- Still the defect have not been resolve by developer
- Developer & test & deployment to client if defects have not been fixed in Saturday & Sunday work.
- Then we will inform to PM these defect will take more time. PM will inform throw Mail to client and these defects will be fixed in next sprint.
- In next sprint PM will create US **– "Carried over Bug".**

17. What problem you have faced in your carrier? OR What problem you have faced in your last project?

- If we have less knowledge about project domain.
- If we have less time for testing.
- If we have less resources / less tester.
- If developer is not communicate properly/ developer making silly mistake.
- If we have problem in environments.
- If we have frequent changes in requirements.

18. What is product backlog?

- The product backlog is a kind of bucket or source where all the user stories are kept.
- This is maintained by the product owner.
- The product backlog can be imagined as a wish list of the product owner who prioritizes it as per the business needs.
- During the planning meeting, one user story is taken from the product backlog, then the team does the brainstorming, understands it and refines it and collectively decides which user stories to take, with the intervention of the product owner.

19. **What is sprint backlog?**

- Based on the priority, user stories are taken from the Product Backlog as one at a time.
- The Scrum team brainstorms on it determines the feasibility and decides on the stories to work on a particular sprint.
- The collective list of all the user stories which the scrum team works on a particular sprint is known as Sprint backlog.

20. What is story points?

- Story points are a quantitative indication of the complexity of a user story.
- Based on the story point, estimation and efforts for a story are determined.
- A story point is relative and not absolute. In order to make sure that our estimate and efforts are correct, it's important to check that the user stories are not big. The more precise and smaller is the user story, the more accurate will be the estimation.
- Each and every user story is assigned to a story point based on the Fibonacci series (1, 2, 3, 5, 8, 13&21). Higher is the number, the complex is the story.
- **To be precise**
  - If you give 1 / 2 / 3 story point it means that the story is small and of low complexity.
  - If you give points as 5 / 8, it is a medium complex and
  - 13 and 21 are highly complex.

21. **How do you calculate a story point?**

- A story point is calculated by taking into consideration the development effort+ testing effort + resolving dependencies and other factors that would require to complete a story.

22. **What are impediments?**

- Any hindrance which prevents the smooth flow of work or due to which the team is not able to perform its task in a better way is what we call **'impediments'.**

23. What are the principles of agile testing?

- **Some major principles of agile testing are:**
  - Customer satisfaction.
  - Bug-free clean code.
  - Changes are welcome by customer.
  - Whole team, business people and developers work collectively.
  - Instead of lengthy documentation, focus on the essence.
  - It focuses on face to face conversation.
  - It promotes sustainable development.

24. **What are the main roles in the scrum?**

- **Scrum Team**: Scrum team is made by an individual person who works collectively to achieve a particular task. The team works in a bond to deliver committed and requested products.
- **Scrum Master**: Scrum Master is responsible for the proper execution or working of the scrum team. Being a servant – leader and a coach, he ensures the proper productivity of a team towards scrum sprint goal.

- **Product Owner**: The product owner has the responsibility to deliver a complete picture of what to build and to convey that idea to the team.

25. What are the different artifacts in scrum?

- **There are three scrum artifacts:**
  - **Product Backlog:**
    - The product backlog includes all the items that will be delivered for the product.
    - This list is constantly evolving.
    - The commitment of the product backlog is the product goal.

  - **Sprint Backlog:**
    - Sprint backlog is a list of all items committed to being delivered within a sprint.
    - Once decided, the sprint backlog cannot be changed.
    - The commitment of the sprint backlog is the sprint goal.

  - **Product Increment:**

    - Product increment is usually delivered at the end of the sprint, which is a workable increment of the overall product.

26. **Explain the term 'increment'?**
- When the team finishes the sprint, hopefully they have completed everything they forecasted.
- The sum of all the product backlog items which were completed in a sprint is called **increments.**
- This new increment also has the value of increment of the previous sprint.

Questions on Test Execution?

1. **What is your approach, when SIT environment is crash?**
   - If client want Deployment with the sprint and SIT environment is crash, then testerwill access the UAT environment.
   - UAT environment access throw remote desktop.

2. **What unit testing documents will contains?**
   - Unit testing documents which contains screenshot for testing white box testing, table's name, URI/URL, etc.
3. **Who is responsible for writing test cases?**
   - Test cases will be written by tester
   - While preparing test cases, we will consider:
     - Business logic related test cases
     - Input domains related test cases
     - UI/GUI related test cases.

4. **What are the different defect reporting tool do you know?**

   - There are number of defect reporting tools available in market like *Clear Quest, Dev Track, Jira,* ***Quality Center, Bug Zilla etc.***

5. **What are different field we have in excel sheet while writing the test case?**

   - **Test Case:**
     - Step by step actions to be performed to validate functionality of AUT (How to test).
     - Test case contains test steps, expected result & actual result.
     - **Test Case Contents:**
       - Sr. No.
       - Test case id:
       - Priority
       - Reference
       - Test scenario.
       - Pre-requisite.
       - Test data.
       - Test steps.
       - Expected result
       - Actual result.
       - Pass/fail.

6. **What is defect resolution?**

   - After receiving the defect report from the testing team, development team conduct a review meeting to fix defects. Then they send a resolution type to the testing team for further communication is known as **defect resolution.**
   - Resolution Types: -
     - Accept
     - Reject
     - Duplicate
     - Enhancement
     - Need more information
     - Not Reproducible
     - Fixed
     - As Designed.

7. **What you have to maintain while executing the test cases?**

- As per test cases we have to test functionality.
- While test case execution, tester will prepared the test proof.
- In TCE, if we found defects then raised/ create in JIRA/ TFS tool.
- Inform to developer throw email (JIRA/ TFS).
- Test proof will contains- Test cases against screen-shot.

8. **What are the different report? Have you involved in preparation of report?**

- In my project, during test case executions tester maintains **test proof.**
- We will also maintain defects which contains all defects related to user stories.
- Test summary report is prepared by test lead which contains all records such as test case design, test case executions details, test proof, defect report etc. of current sprint
- Test closure report is also prepared by test lead. It indicate sign-off from the testing team.

9 **What is roles & responsibility?**

- Requirement analysis / US analysis/ understand.
- Identify Test scenario against US.
- Test cases writing against Test scenario.
- Test cases review (Internal/ external review).
- Test cases execution (Test proof).
- Defect raised/ created.
- Defect report against current sprint.
- Test summery report (Helping to Test lead).
- Client interaction (Sprint review meeting- For respective US we will demo/ review).

10. **Which parameter consider in Test cases review?**

- Test case should be simple.
- Test cases should be understandable.
- Test cases should be functionality against the US
- Test cases should be business logic related test cases
- Test cases not should be duplicated.
- Test cases should follow standard format.
- Test cases should grammatically correct.

11. When you will know that testing is completed?

- All test cases execution should be completed,
- All defect should be fixed & tested (retesting & regression testing).
- Tractability matrix should be completed.

12. How many test cases you will write per day?

a. How many test cases you will write per US?

- It totally depends on complicity of US.
- User story >> in an average, I will write 25 to 30 TCD/ US.

13. How many automation test script you will write per day?

- In an average, I will write 5 to 6 test script.

14. What are testing techniques for test cases writing?

- **Test techniques/ coverage for writing the test cases**
  - Behavioral testing techniques / coverage
  - Input domain testing techniques / coverage (BVA, ECP, Decision)
  - Error handling testing techniques / coverage
  - Backend testing techniques / coverage
  - Service level testing techniques / coverage
  - Calculation based testing techniques / coverage

15. What problem you have faced in your carrier? OR What problem you have faced in your last project?

- If we have less knowledge about project domain.
- If we have less time for testing.
- If we have less resources / less tester.
- If developer is not communicate properly/ developer making silly mistake.
- If we have problem in environments.
- If we have frequent changes in requirements.

16. What is your approach, if developer is not accepting the defect?

- If tester will found the defects, then tester will raised the defect in JIRA/ TFS tool.
- While creating the defects, screenshot will attachment with defects & inform throw the mail.
- We will take call with developer & share the screen with developer and shows the defects.
- Tester will inform to Test lead & PM throw mail.
- Same we will inform in daily stand up.

17. What is your approach, if a defect has not occurred in Dev environment but present in SIT environment?

- If tester will found the defects, then tester will raised the defect in JIRA/ TFS tool.
- While creating the defects, screenshot will attachment with defects & inform throw the mail.
- We will take call with developer & share the screen with developer and shows the defects.
- Tester will inform to Test lead & PM throw mail.
- Same we will inform in daily stand up.
- Problem of these Defect – Deployment/ Environment.
- These defects will be occurred due to Configuration file.

18. Do you run all regression tests for every bug fixed?

- No, I didn't run regression test cases for every bug fixed. I run regression tests once for every build.

19. When you fill the data in the application form, how do you ensure that the data is stored in the correct tables and columns?

- We can write an SQL query to retrieve data from the data base and compare the query result with the data we have filled in the application forms.

20. What are the entry and exit criteria for test execution?

- **Entry criteria:**
  - Coding should be completed
  - Test cases should be ready and base lined
  - RTM should be updated
  - Test data should be ready and base lined
  - Test environment/set up should be ready.
  - S/w tools should be ready and approved.

- **Exit criteria:**
  - All test cases must be executed and passed
  - All defects identified must be fixed, retested and closed.
  - Test execution summary report must be prepared.

21. What is test case?

- Test case is a set of inputs, conditions and expected outcomes which a tester will deter- mine whether an application is working correctly or not.

22. What is test log?

- It is a report of what tests have been executed and their status like pass/ fail.
- It is also known as Test execution report.

23. How do you know for which functionalities you should write test case?

- My lead writes top level requirements in QC and assigns to each team member.
- We divide test requirements further into sub requirements.
- Then we identify test conditions for each sub requirement and create test cases. Test cases are reviewed after that. Reviewed and approved test cases will go to ready state.

24. How do you know your test cases are completed?

- We follow two step approaches to ensure that test cases are completed.
- **Reviews--** it ensures that quality of the test cases is good.
- **Requirement traceability matrix ---**it ensures that all requirements have been covered through test cases.

25. What is the percentage of positive and negative test cases that you write?

- Approx. 30% positive and 70% negative.

26. How do you check broken links?

- Many tools are available for this. We use tools like menu.

27. How do you find whether a test case is a good test case or bad test case?

- A good test case is one which finds the bug or one which has a high probability of finding the bug.
- A good test case should be documented clearly, so that it can be executed by anyone without any difficulties and confusion.

28. Do you update the test cases after receiving build based on the application screen?

- During execution, if we feel any test case requires an update, we will do it with the approval of the team lead. But this work is very limited.

29. What are the exit criteria for test closure?

- Checking whether planned deliverables have been delivered.
- Finalizing and archiving test ware.
- Handover of test ware for maintenance.
- Analyzing lessons learned for improvement of test maturity.
- Testing sign off.

30. Should you understand the whole project functionality or only the functionality assigned to you?

- I should have a big picture of the whole project.
- In other words I should have an over- view of the whole project and detailed screen and field level understanding of the assigned functionalities.

31. What parameters do the test manager considers to take the decision to stop testing?

- The important parameters a test manager looks into are:
  - Whether all requirements have been developed or not.
  - Whether all requirements have been covered through testing.
  - Whether all requirements have been handled through fixed or differed status.

32. What is an entry criterion for test closure?

- Decision to stop testing

33. What is test ware?

- Test ware is Artifacts produced during the testing process.
- Test ware include test cases, test plan, automation scripts, test data, test environment set-up and clear up procedures and any additional software or utilities used in testing.

34. What is lessons learnt document?

- No of test cases/scenarios blocked.
- No of defects verified and their respective status.
- Weekly status reporting.
- Test case summary.
- Issues found.
- Issues resolved.
- Critical issues which are still open and which requires immediate attention from the client side.
- The report should also contain high plan for the next week.

35. What is web server log?

- Every time a web page is requested, the webserver automatically logs the following in- formation.
- The IP address of the visitor.
- Date and time of the request.
- The URL of the requested file.
- The URL, the visitor came from immediately before.
- The visitors web browser type and operating system.

36. How many test cases can you execute per day?

- It depends on the size and complexity of the test cases. Approximately i.e. execute around 50 test cases per day which comes to 40 pages approx.

37. What is code review?

- Code review is the process of reviewing the code written.

- Code reviews are conducted for the code developed by the developer and also for the automation scripts developed by the automation engineer.

38. How do you say review was successful?

- If every reviewer prepares well before the review and provides good comments for improvement of the work product, we can say that the review was successful.

39. Explain different test execution strategies?

- There are 3 test execution strategies.
- They are pass1, pass2, pass 3

- **Pass1 Test execution strategy:**
  - In this execution model one execution cycle will be there.
  - In this one execution cycle. Itself testers log defects and retest that defect.
  - This is useful in stable, small with.

- **2nd pass:**
  - Development team releases new build claiming all the defects are fixed.
  - Testing team retest all defects with Adhoc regression.
  - If new defects are found development team release new build and the life cycle is repeated until no new defects.

- **3rd pass:**
  - Testing team runs full regression suite and this phase completes only when full regression is completed.
  - This is good model for large, complex and critical projects.
  - In case of getting large no of defects in pass -2 strategy, one may have to move to pass-3 strategy.

40.Test Cycle Closure

- **Activities:**
  - Evaluate cycle completion criteria based on Time, Test coverage, Cost, Software,
  - Critical Business Objectives, Quality.
  - Prepare test metrics based on the above parameters.
  - Document the learning out of the project
  - Prepare Test summary report
  - Qualitative and quantitative reporting of quality of the work product to the
  - Customer.
  - Test result analysis to find out the defect distribution by type and severity.
  - Deliverables
    - Test Closure report
    - Test metrics

41. How much information you can review in one day?

- Per hour we review around 20 pages if it is documentation and 200 lines if it is code.

42. Explain what do you document during the review process?

- We document page and line number of defect, origin of defect, severity of defect.
- We also document other information like work product ID, reviewers, etc.

43. What are the roles present in the review?

- **Manager:**
  - Decides on execution of reviews.
  - Allocates time in project schedules.
  - Determines if the review objectives have met.
- **Moderator:**
  - Leads the review, including planning and running the meeting.
  - Follows-up after the meeting.
- **Author:**
  - The author is the person who has created the item to be reviewed.
  - The author may also be asked questions within the review.
- **Reviewer:**
  - The reviewer are the attendees of the review who attempt to find errors in the item under review.
  - They should come from different perspectives in order to provide a well-balanced review of the item
- **Scribe:**
  - The scribe or recorder is the person who is responsible for documenting issues raised during the process of the review meeting.

44. Explain Test Matrix?

- **Defect Density: Number of defects identified per requirement/s**
  No.of defects found / Size(No. of requirements)
- **Defect Removal Efficiency (DRE):**
  (A / A+B) * 100
  (Fixed Defects / (Fixed Defects + Missed defects)) * 100
  - A- Defects identified during testing/ Fixed Defects
  - B- Defects identified by the customer/Missed defects
- **Defect Leakage:**
  (No.of defects found in UAT / No. of defects found in Testing) * 100
- **Defect Rejection Ratio:**
  (No. of defect rejected /Total No. of defects raised) * 100
- **Defect Age:** Fixed date-Reported date
- **Customer satisfaction** = No.of complaints per Period of time

45. **Explain defect related Matrix?**

- **% of Test cases Executed:**
  No.of Test cases executed / Total No. of Test cases written) * 100
- **% of test cases NOT executed:**
  (No.of Test cases NOT executed/Total No. of Test cases written) * 100
- **% Test cases passed**
  (No.of Test cases Passed /Total Test cases executed) * 100
- **% Test cases failed**
  (No.of Test cases failed / Total Test cases executed) * 100
- **%Test cases blocked**
  (No.of test cases blocked / Total Test cases executed) * 100